

Introduzione

In Haskell tutta l'informazione necessaria per l'esecuzione risiede all'interno dell'espressione stessa in ogni momento, senza dipendere da stati esterni o variabili invisibili.

📄 Trasparenza Referenziale

Nei *linguaggi imperativi*, una variabile rappresenta un **indirizzo di memoria** il cui contenuto può cambiare nel tempo. Nei linguaggi funzionali, il *binding* (legame) avviene tra un nome e un valore immutabile. Se consideriamo l'equazione della differenza simmetrica di insiemi:

$$A \otimes B = (A \setminus B) \cup (B \setminus A)$$

Ogni occorrenza di A (e B) denota e *denoterà sempre lo stesso esatto insieme*. Questa proprietà, per cui un'espressione può essere sempre sostituita con il suo valore valutato senza alterare il comportamento del programma, è detta trasparenza referenziale. Elimina totalmente gli effetti collaterali e permette un ragionamento algebrico sul codice.

In Haskell utilizziamo la notazione:

```
f :: X → Y
```

significa che f è una funzione e il *dominio* è di tipo X , mentre il *codominio* è Y .

Oltre ai tipi predefiniti (`Integer`, `Float`, `Char`), Haskell basa gran parte della gestione dati sui costruttori di tipo:

- **Liste**: Sequenze omogenee gestite sequenzialmente, es. `[3, 7, 11] :: [Integer]`.
- **Le stringhe** ("tpfi") non sono altro che syntactic sugar per liste di caratteri `['t', 'p', 'f', 'i']`.
- **Tuple**: N-uple di elementi anche disomogenei, utili per restituire risultati multipli da una funzione, es. `(3, 'c') :: (Integer, Char)`

Prelude è l'ambiente standard caricato quando l'interprete inizia a girare, contiene funzioni, costanti, tipi, etc...

🔗 Composizione di funzioni

```
(.) :: (b → c) → (a → b) → a → c
```

Operatore di composizione funzionale, analogo al $\circ$ matematico, è una funzione di ordine superiore perché ha come parametro altre funzioni.

In Haskell possiamo *definire le funzioni* dando un nome a un'espressione oppure utilizzando le lambda-espressioni (funzioni anonime). Un esempio fondamentale è la funzione identità, definibile come `id x = x` oppure `id = \x → x`.

Haskell è **statically type-safe**, ossia che se un programma viene tipato correttamente, non vi sarà nessun errore di tipo durante l'esecuzione.

Le espressioni non tipabili

Tutti i lambda termini *tipabili* in haskell *terminano*.

❗ Omega

La seguente espressione non è tipabile.

```
omega = \x → x x
```

La funzione prende un parametro `x` e non sapendo cosa sia gli assegniamo una variabile di tipo α . Nel corpo della funzione stiamo prendendo `x` e lo applichiamo a `x`, quindi il primo `x` deve essere per forza una funzione. Essendo tale, deve avere come tipo $\beta \rightarrow \gamma$. Quindi la funzione si aspetta un parametro β in input ma noi sappiamo che `x` ha tipo α . Quindi $\alpha = \beta$, di conseguenza sostituendo il tipo del primo `x` diventa $\alpha \rightarrow \gamma$.

Ma se noi avevamo definito inizialmente `x` con tipo α e poi $\alpha \rightarrow \gamma$, essendo `x` la stessa identica variabile non può avere due tipi diversi, allora si cerca di unificarli a $\alpha = \alpha \rightarrow \gamma$.

Ottenendo ciò si creerebbe un loop perché ad ogni occorrenza di α si andrebbe a sostituire

$$\alpha = (((\alpha \rightarrow \gamma) \rightarrow \gamma) \rightarrow \gamma) \rightarrow \gamma$$

Si possono comunque definire *funzioni ricorsive che non terminano*, ad esempio:

```
omega' x = omega' x.
```

Il compilatore inferisce per questa funzione il tipo $t \rightarrow t1$. Ma quale funzione può accettare un parametro di un qualsiasi tipo e restituire un risultato di un altro qualsiasi tipo scorrelato? L'unica spiegazione è che la *funzione non restituisca mai nulla* perché entra in un loop infinito. In programmazione funzionale, la semantica di un'equazione ricorsiva è identificabile come il punto fisso della trasformazione che la definisce.

Espressione Let

```
let x = \x → x in x x
```

è tipabile in Haskell poiché verrà prima valutato `x` come $\forall t. t \rightarrow t$, poi il primo `x` può essere specializzato in $(t \rightarrow t) \rightarrow (t \rightarrow t)$ e il secondo in $(t \rightarrow t)$.

🔄 L'oggetto indefinito: undefined

In Haskell esiste una costante predefinita chiamata `undefined`. Se ne esploriamo il tipo, otteniamo $\forall a. a$. Questo tipo è "inconsistente" da un punto di vista logico: possiede il tipo più generale in assoluto, e dunque *appartiene a tutti i tipi contemporaneamente*. È la rappresentazione canonica di computazioni indefinite, fallimenti previsti o loop infiniti.

Costrutti, Pattern Matching e Tuple

La logica di controllo si basa fortemente sul *pattern matching*. Nei linguaggi funzionali, il ramo else del costrutto `if` non è opzionale: un `if` è un'espressione che deve sempre restituire un valore valutabile. Un'alternativa più pulita sono le *espressioni guardate* (`| condizione = risultato`), le quali vengono valutate sequenzialmente.

Il pattern matching funziona perfettamente anche sui valori strutturati come le *Tuple*. Funziona da vero e proprio "distruttore", permettendo di estrarne i componenti interni direttamente nella firma della funzione. Scrivendo le funzioni predefinite *fst* e *snd*:

```
myFst (x , y ) = x
mySnd (x , y ) = y
```

❶ Pillole di Laziness: Il comportamento di null

La valutazione pigra (*lazy evaluation*) mostra i suoi effetti anche su funzioni semplici come `null`. Questa funzione è considerata "stretta" (strict) solo nella misura in cui ha bisogno di guardare il primo nodo per capire la struttura. Se eseguiamo `null undefined`, il programma va in crash (`Exception: Prelude.undefined`). Ma se eseguiamo `null (undefined : undefined)`, il risultato sarà False! Haskell non ha bisogno di valutare il contenuto della lista (che è indefinito); gli basta *vedere che esiste un costruttore* `:` per dedurre matematicamente che la lista non è vuota.

❶ Concetto Chiave: L'As-Pattern per le ottimizzazioni

Quando manipoliamo liste complesse, capita spesso di fare pattern matching per estrarre la testa e la coda, ma allo stesso tempo avere bisogno di passare alla successiva chiamata ricorsiva l'intera lista intatta. Ricostruire la lista scrivendo `x:xs` richiede l'allocazione di un nuovo nodo `cons` in memoria. Per evitare questo spreco, Haskell offre l'*as-pattern*, indicato con il simbolo `@`. Scrivendo `xs@(x:txs)`, stiamo dicendo: "smonta la lista in `x` e `txs`, ma memorizza un riferimento all'intera lista originale nella variabile `xs`".

Merge

```
merge [] ys = ys
merge xs [] = xs
merge xs@ ( x : txs ) ys@ ( y : tys )
  | x <= y = x : merge txs ys
  | otherwise = y : merge xs tys
```

Zip e Concat

- *zip*: Prende due liste e le "cuca" insieme, creando una singola lista di tuple. Si ferma automaticamente non appena la lista più corta si esaurisce.

```
myZip ( x : xs ) ( y : ys ) = ( x , y ) : myZip xs ys
myZip _ _ = [] -- Si ferma se una delle due è vuota
```

- *concat*: "Scioglie" una lista di liste in un'unica lista piatta, giustapponendo tutti i valori interni in ordine.

```
yConcat ( xs : xss ) = xs ++ myConcat xss
myConcat [] = []
```

Funzionali e Applicazioni

Per evitare di scrivere manualmente liste sequenziali, Haskell fornisce delle comode notazioni chiamate enumerazioni:

- `[m..n]`: genera la lista finita da m a n. Esempio: `[1..5]` genera `[1,2,3,4,5]`.
- `[m..]`: genera una lista infinita a partire da m. Grazie alla *lazy evaluation*, possiamo scrivere `[0..]` per indicare tutti i numeri naturali, senza bloccare la macchina.
- `[m, n..p]`: genera una lista finita in cui il "passo" (la distanza tra gli elementi) è definito dalla differenza tra n e m. Ad esempio, `[1, 3..11]` restituisce i dispari `[1,3,5,7,9,11]`.
- `[m, n..]`: come sopra, ma genera una lista infinita (es. `[1, 3..]` genera tutti i dispari esistenti).

Curryficazione

🔗 Concetto Chiave: Isomorfismo della Curryficazione

Da un punto di vista degli insiemi, esiste un isomorfismo garantito (basta fare attenzione all'associatività delle parentesi):

$$A \times B \rightarrow C \sim A \rightarrow (B \rightarrow C)$$

Il grande vantaggio delle versioni curryficate è l'applicazione parziale. Passando un solo argomento a una funzione binaria, otteniamo in ritorno una funzione unaria specializzata, perfettamente compatibile con i funzionali di ordine superiore.

Haskell fornisce tre piccoli funzionali predefiniti che aiutano a convertire e manipolare la firma delle funzioni:

```
-- Trasforma una funzione su tupla in una funzione curryficata
myCurry f x y = f (x, y)
-- Tipo : ((a, b) -> c) -> a -> b -> c

-- Trasforma una funzione curryficata in una che accetta una tupla
myUncurry f (x, y) = f x y
-- Tipo : (a -> b -> c) -> (a, b) -> c

-- Inverte l'ordine dei due parametri di una funzione
myFlip f a b = f b a
-- Tipo : (a -> b -> c) -> b -> a -> c
```

Map

La funzione `map` applica una trasformazione a tutti gli elementi di una lista. Gode di proprietà algebriche notevoli, come `map id = id` (mappare l'identità restituisce la lista invariata) e `map (f . g) = map f . map g` (mappare la composizione di funzioni equivale a comporre le mappe).

List comprehension

Un costrutto sintattico estremamente amato per manipolare le liste è la *List Comprehension*, che ricalca la definizione matematica degli insiemi (set comprehension), come $x \in A | P(x)$. In Haskell si scrive: `1 > [ x ^2 | x <- [1..5]]` `2 [1, 4, 9, 16, 25]`

Altri operatori

`zipWith` prende in input una funzione a due argomenti e due liste. Il suo compito è scorrere le due liste in parallelo e applicare la funzione alle coppie di elementi corrispondenti (il primo con il primo, il secondo con il secondo, ecc.), restituendo una nuova lista.

```
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

```
-- Esempio
```

```
zipWith (+) [1, 2, 3] [10, 20, 30]
```

```
-- Risultato: [11, 22, 33]
```

`zapp` prende una lista di funzioni e una lista di valori, applicando l'i-esima funzione all'i-esimo valore.

```
zapp :: [a -> b] -> [a] -> [b]
```

```
zapp [(+1), (*2), (^2)] [10, 20, 30]
```

```
-- Risultato: [11, 40, 900]
```

`iterate` prende una funzione, un valore di partenza (il seme), e costruisce una lista infinita applicando la funzione al risultato precedente.

```
iterate :: (a -> a) -> a -> [a]
```

Concetto Chiave: Il significato di $\Rightarrow$ e le Classi

Tutto ciò che si trova a sinistra della freccia $\Rightarrow$ rappresenta il vincolo di classe (class constraint). Il tipo `a` non può essere un tipo qualsiasi, ma deve contemporaneamente appartenere alla classe `Ord` (perché usiamo `<` e `==`) e alla classe `Num` (perché usiamo la sottrazione `-`). Le Type Classes di Haskell sono simili alle Interfacce in Java: impongono che un tipo implementi un insieme specifico di funzioni o operatori per essere considerato un'istanza valida di quella classe.

Concetto Chiave: La liberazione dal NULL

Nei linguaggi imperativi, le operazioni che falliscono restituiscono spesso puntatori nulli (`null`), causando i famigerati crash a runtime. In Haskell, per codificare il concetto di fallimento o assenza di valore, si utilizza in modo type-safe il tipo polimorfo `Maybe`.

```
data Maybe a = Just a | Nothing
```

Dimostrazioni in haskell

Ricorsione su Liste

I funzionali di riduzione astraggono gli schemi di ricorsione sulle liste.

- **foldr** (fold right) sostituisce i costruttori in loco e raccoglie i risultati partendo dal caso base a destra:

```
-- Foldr :: (a -> b -> b) -> b -> [a] -> b
Foldr f v (x:xs) = f x (Foldr f v xs)
foldr (#) v [x, y, z] = x # (y # (z # v))
```

- **foldl** (fold left) spinge i calcoli in avanti durante la discesa, fungendo da iteratore con accumulatore:

```
-- Foldl :: (b -> a -> b) -> b -> [a] -> b
Foldl f v (x:xs) = Foldl f (f v x) xs
foldl (#) v [x, y, z] = ((v # x) # y) # z
```

⚡ Proprietà di foldr: La Distributività

Un teorema potentissimo afferma che foldr distribuisce un operatore # sull'operatore di concatenazione ++:

```
foldr f e (xs ++ ys) = foldr f e xs # foldr f e ys
```

Caso Base (xs=[]):

→ foldr f e ([] ++ ys) $\xrightarrow{\text{Def. conc}}$ foldr f e ys

← foldr f e xs # foldr f e ys → e # foldr f e ys, il che è vero sse e è l'elemento neutro per #

Caso Induttivo (x:xs)

→ foldr f e ((x:xs) ++ ys)

$\xrightarrow{\text{Def. conc}}$ foldr f e (x:(xs ++ ys))

$\xrightarrow{\text{Def. Foldr}}$ f x (foldr e (xs ++ ys))

$\xrightarrow{\text{Def. Ipotesi}}$ f x (foldr f e xs # foldr f e ys)

← foldr f e (x:xs) # foldr f e ys

$\xrightarrow{\text{Def. conc}}$ f x (Foldr f v xs) # foldr f e ys

Affinché questa equazione sia sempre valida, è stato dimostrato per induzione che devono sussistere due condizioni stringenti:

- e deve essere l'elemento neutro (sinistro) dell'operatore #
- Deve valere che $f x (y \# z) = (f x y) \# z$ (che è sempre vero se $f = \#$ ed # è associativo).

Da questo teorema deriva istantaneamente che $\text{sum } (xs ++ ys) = \text{sum } xs + \text{sum } ys$, semplicemente applicando il fatto che sum è definita come foldr (+) 0.

Haskell essendo *lazy*, valutando foldl non esegue subito le somme matematiche ma alloca nella memoria un intero blocco non valutato, cominciando la computazione reale quando la ricorsione termina. Si ottiene un pesante **overhead** di memoria.

Al contrario, foldr restituisce subito il primo costruttore valutabile.

Lambda Calcolo

La sua Sintassi è definita dalla seguente grammatica, che genera i cosiddetti **λ -termini** (M, N):

$$M, N ::= x \mid \lambda x. M \mid (MN)$$

- **Variabile** (x): Rappresenta un parametro o un valore.
- **Astrazione** ($\lambda x. M$): Crea una funzione anonima che lega la variabile x nel corpo M .
- **Applicazione** ($(M N)$): Rappresenta la chiamata della funzione M sull'argomento N .

Per alleggerire le formule, si adottano convenzioni sintattiche rigorose:

- L'**Applicazione associa a sinistra**: $MNP \equiv ((MN)P)$.
- L'**Astrazione associa a destra**: $\lambda xyz. M \equiv \lambda x. (\lambda y. (\lambda z. M))$. Questa proprietà permette di rappresentare funzioni a più argomenti valutandole un argomento alla volta (Curryficazione).

Le variabili libere

L'ambiente di un termine è determinato dalle sue variabili. L'insieme delle **Variabili Libere** (M) è definito induttivamente:

- $(x) = x$
- $(\lambda x. M) = (M) x$
- $(MN) = (M) \cup (N)$

Alpha e Beta

⚡ La α -conversione e la β -riduzione

Il nome di una variabile legata è ininfluente: $\lambda x. x$ è identico a $\lambda y. y$. Questa equivalenza si chiama **α -conversione**. Si assume sempre la **convenzione di Barendregt**: i nomi delle variabili legate vengono scelti tutti diversi tra loro e diversi da quelli delle variabili libere.

$$\lambda x. M =_{\alpha} \lambda y. [yx]M$$

Dove y non è una variabile libera in M .

Un termine senza β -redex, ossia che non ha più operazioni da eseguire, è detto in **forma Normale**. Ed è un valore:

$$(\lambda x. M)N \rightarrow_{\beta} M[Nx]$$

Operatori che vanno in loop

- $\omega = \lambda x. xx$
- $\Omega = \omega\omega = (\lambda x. xx)\omega = \omega\omega$
in un linguaggio imperativo è paragonabile a `void f () { return f(); }` perché va in loop
- $\omega_3 = \lambda x. xxx$

- $\Omega_3 = \omega_3 \omega_3 = (\lambda x. xxx) \omega_3 = \omega_3 \omega_3 \omega_3 = \Omega_3 \omega_3$

In un linguaggio imperativo è paragonabile a `void f() {malloc(1); return f();}`

Combinatori e Strutture Dati Logiche

$((M) =)$ sono detti *combinatori*. In λ -calcolo i dati primitivi non esistono: logica e strutture si codificano come puro flusso di controllo.

Definiamo i valori di verità come funzioni di scelta:

- *Vero* / K : $\equiv \lambda xy. x$ (*Cancellatore*: restituisce il primo e ignora il secondo).
- *Falso* / $:$: $\equiv \lambda xy. y$ (Restituisce il secondo e ignora il primo).
- $\equiv \lambda xyz. xz(yz)$
O è derivato da S e T

L'istruzione `if x then M else N` è l'applicazione `x M N`. Operatori logici:

- *NOT*: $\lambda x. x$
- *AND*: $\lambda xy. xy$

Predecessore

Nel lambda calcolo, la funzione predecessore sui numerali di Church si definisce tipicamente usando una funzione ausiliaria (qui chiamata `sk`) che lavora su coppie, traslando i valori in avanti:

$$sk(m, n) \rightarrow (n, n + 1)$$

Partendo da $(0, 0)$, applicando n volte la funzione, otteniamo:

$$(0, 0) \rightarrow (0, 1) \rightarrow (1, 2) \rightarrow \dots$$

$$n \text{ olte } sk(0, 0) \rightarrow (n - 1, n)$$

Da cui si può estrarre il primo elemento per avere il predecessore $n - 1$.

Costrutto If-Then-Else (Booleani di Church)

La logica condizionale `if x then M else N` si codifica applicando il booleano x ai rami M ed N . I booleani sono definiti come funzioni di due argomenti:

- Vero (T): sceglie il primo argomento $\lambda xy. x$
- Falso (F): sceglie il secondo argomento $\lambda xy. y$

$$M N = M \implies (\lambda xy. x) M N \rightarrow (\lambda y. M) N \rightarrow M$$

$$M N = N \implies (\lambda xy. y) M N \rightarrow (\lambda y. y) N \rightarrow N$$

Tuple (Coppie)

Una coppia (M, N) è codificata come una funzione che prende un selettore x e gli applica M ed N :

$$(M, N) \equiv \lambda x. x \ M \ N$$

Per estrarre gli elementi, si passano i booleani true (per il primo) e false (per il secondo) come selettori:

- Estrazione del 1° elemento:

$$(\lambda x. x \ M \ N) \rightarrow M \ N \rightarrow M$$

- Estrazione del 2° elemento:

$$(\lambda x. x \ M \ N) \rightarrow M \ N \rightarrow N$$

n-uple

Il concetto di coppia può essere esteso a strutture di k elementi:

$$(M_1, \dots, M_k) \equiv \lambda x. x \ M_1 \dots M_k$$

Per selezionare l' i -esimo elemento, servono k selettori del tipo $(\lambda y_1 \dots y_k. y_i)$:

$$(\lambda x. x \ M_1 \dots M_k)(\lambda y_1 \dots y_k. y_i) = M_i$$

Ricorsione Primitiva sui Naturali

Formulazione della ricorsione primitiva tramite la funzione **rec**, un caso base e un caso ricorsivo:

Caso Ricorsivo:

$$rec\ f\ n = f\ n\ (rec\ f\ (n\ 1))$$

Caso Base:

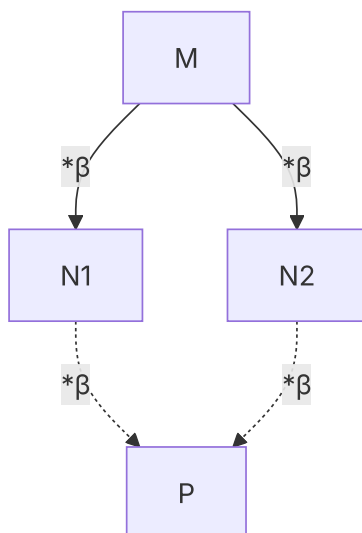
$$rec\ f\ 0 =$$

Teorema di Church-Rosser (o Confluenza)

Il teorema afferma che l'ordine delle riduzioni non influisce sul risultato finale. Se un termine può essere ridotto in due modi diversi, esiste sempre un modo per far ricongiungere i due percorsi.

Definizione Formale: Se $M \rightarrow_{\beta} N_1$ e $M \rightarrow_{\beta} N_2$ (ossia può essere derivato attraverso due vie diverse)

$\Rightarrow$ P.t.c. $N_1 \rightarrow_{\beta} P$ e $N_2 \rightarrow_{\beta} P$ (con P in comune nelle derivazioni)

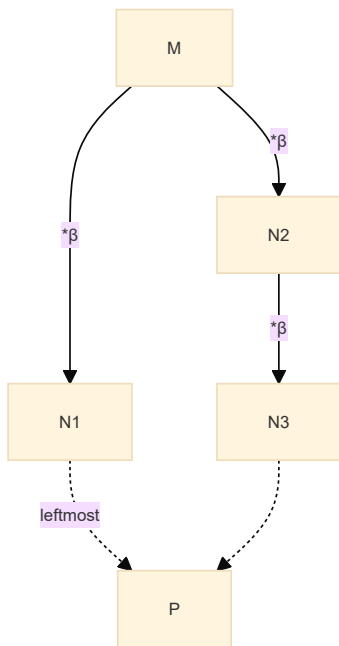


Teorema di Standardizzazione

Questo teorema ci dice quale strategia usare per essere sicuri di arrivare al risultato, se questo esiste.

Definizione: Se esiste una computazione terminante $M \rightarrow P$ (con P in forma normale, ossia irriducibile)

$\Rightarrow$ sicuramente termina la riduzione leftmost / outermost (termina la computazione "call by name").



(Nel diagramma, il percorso che passa per N_1 rappresenta la strategia leftmost, che porta direttamente alla forma normale P , mentre le altre strategie come $N_2 \rightarrow N_3$ potrebbero fare passaggi aggiuntivi o rischiare di non terminare).

Aritmetica di Church

⚡ Definizione

I numeri naturali vengono codificati come iteratori. Il numerale di Church n applica una funzione s per n volte a una base z :

$$n \equiv \lambda sz. s(s(\dots(sz)\dots))$$

- $0 \equiv \lambda sz. z$ (Equivale al combinatore F)
- $1 \equiv \lambda sz. sz$

Le operazioni manipolano le iterazioni:

- **Successore**: $su \equiv \lambda xsz. s(xsz)$.
- **Addizione**: $add \equiv \lambda mnsz. ms(nsz)$.
- **Moltiplicazione**: $mul \equiv \lambda mns. m(ns)$.

Combinatori di Punto Fisso

Il **punto fisso** è un parametro che se passato ad una funzione, ritorna il parametro stesso: $f(x) = x$
 Per supportare la **ricorsione generale** (funzioni anonime che richiamano se stesse), il λ -calcolo usa il **Combinatore di Punto Fisso Y**, il quale garantisce $YM \rightarrow M(YM)$.

- **Church (YC)**: $\lambda f. (\lambda x. f(xx))(\lambda x. f(xx))$.
- **Turing (YT)**: $dove \equiv \lambda xy. y(xxy)$.

Per implementare una funzione ricorsiva come il fattoriale $f = \text{orpo}(f)$, in λ -calcolo non si fa altro che astrarre la $f : \lambda f. \text{orpo}(f)$ e passarvi un combinatore di punto fisso: $Y(\lambda f. \text{orpo}(f))$, innescando l'infinito nastro della ricorsione pura.

Tipaggio in Haskell: Il Sistema Hindley-Milner

Il λ -calcolo puro non ha tipi, permettendo forme divergenti come $\Omega \equiv (\lambda x. xx)(\lambda x. xx)$. Haskell adotta il sistema di tipi *Hindley-Milner*, diviso in:

- *Monotipi* $()$: $::= Int \mid Bool \mid \alpha \mid 1 \rightarrow 2.2$
- *Schemi di Tipo* / Politipi $()$: $::= \mid \forall \alpha.$

Regole di Derivazione

Il giudizio $M :$ indica che sotto l'ambiente Γ , il termine M ha tipo τ . Le regole principali:

Regola della Variabile (*VAR*)

$$(x : \tau) \in \Gamma \quad x : \tau$$

Regola dell'Astrazione (*ABS*)

$$\Gamma \cup \{x : \tau_1\} \quad M : \tau_2 \quad \lambda x. M : \tau_1 \rightarrow \tau_2$$

Regola dell'Applicazione (*APP*)

$$M : \tau_1 \rightarrow \tau_2 \quad N : \tau_1 \quad MN : \tau_2$$

Regola del Punto Fisso (*FIX*)

$$\Gamma \cup \{x : \tau\} \quad M : \tau \quad \text{fi } x. M :$$

Questa regola serve a tipare le definizioni ricorsive native del linguaggio senza passare per il combinatore Y .

Generalizzazione del tipo (*GEN*)

$$M : \tau \quad \alpha \text{ free in } \tau \quad M : \forall \alpha. \tau$$

Specializzazione (*SPEC*)

$$M : \tau \quad \tau \leq \tau' \quad M : \tau'$$

Regola *LET* (Accetta politipi)

$$N : \tau_1 \quad \Gamma \cup \{x : \tau_1\} \quad M : \tau_2 \quad \text{let } x = N \text{ in } M : \tau_2$$

La Fusion Law per foldr

Esiste una forma di induzione generalizzata e preconfezionata, pensata appositamente per il funzionale `foldr`, nota come *Fusion Law*.

Lo scopo della Fusion Law è ottimizzare il codice fondendo due operazioni applicate in sequenza (di cui almeno una è un `foldr`) in un'unica passata sulla lista.

L'equazione generale del teorema che vogliamo dimostrare ha questa forma:

$$f \circ \text{foldr } g \ a = \text{foldr } h \ b$$

Per ricavare le condizioni (i vincoli) che rendono vera l'equazione qui sopra, si procede per analisi dei casi, valutando i tre possibili "stati" in cui può trovarsi una lista in Haskell:

- La lista vuota (`[]`).
- Una lista con almeno un elemento (`x:xs`).
- Una lista divergente o "rotta" (`undefined`).

Funzioni Strette

Nel terzo caso dell'induzione incontreremo il valore `undefined`, che rappresenta una *computazione che va in loop infinito* o genera un'eccezione (un crash). Grazie alla valutazione pigra (lazy) di Haskell, non è detto che passare `undefined` a una funzione faccia crashare il programma. Ad esempio, una funzione costante $k\ x = 5$ ignorerà totalmente il parametro `x`: se chiamiamo `k undefined`, il risultato sarà tranquillamente 5.

Una *funzione stretta* (strict) è invece una funzione che ha assolutamente bisogno di valutare il suo parametro per poter procedere. Se le passiamo un parametro "rotto", lei si rompe a sua volta. Matematicamente, si definisce stretta una funzione f tale per cui:

$$f\ undefined = undefined$$

Ora che abbiamo gli strumenti, applichiamo il ragionamento ai tre casi per ricavare i vincoli della Fusion Law:

1. Caso `[]` (Lista vuota):

- *Sinistra*: Valutiamo `f (foldr g a [])`. Per definizione, `foldr` su lista vuota restituisce il caso base `a`. Otteniamo quindi `f a`.
- *Destra*: Valutiamo `foldr h b []`. Per definizione, otteniamo `b`.

Uguagliando le due parti, otteniamo il primo vincolo: `f a = b`.

2. Caso `(x:xs)` (Passo induttivo):

- *Sinistra*: Valutiamo `f (foldr g a (x : xs))`. Per la definizione di `foldr` induttivo, l'espressione si espande in `f (g x (foldr g a xs))`.
- *Destra*: Valutiamo `foldr h b (x : xs)`. Si espande in `h x (foldr h b xs)`.

Ora usiamo l'ipotesi induttiva: assumiamo che il teorema sia già valido per la coda `xs`. Quindi a destra possiamo sostituire `foldr h b xs` con `f (foldr g a xs)`.

La parte destra diventa: `h x (f (foldr g a xs))`. Se confrontiamo *Sinistra* e *Destra*, notiamo che l'unica differenza è l'operatore più esterno. Sostituendo il blocco ricorsivo (`foldr g a xs`) con una variabile generica `y`, otteniamo il secondo vincolo: `f (g x y) = h x (f y)`.

3. Caso `undefined` (Liste parziali):

- *Sinistra*: Valutiamo `f (foldr g a undefined)`. Poiché `foldr` deve smontare la lista per fare pattern matching, la sua esecuzione su `undefined` crasha, restituendo `undefined`. L'espressione diventa `f undefined`.
- *Destra*: Valutiamo `foldr h b undefined`. Anche in questo caso il risultato è `undefined`. Uguagliando, ricaviamo che deve valere `f undefined = undefined`. Ovvero, il terzo vincolo richiede che f sia una funzione stretta.

⚡ Teorema: Fusion Law per foldr

Siano f, g, h funzioni e a, b valori. Se valgono le seguenti tre condizioni:

1. f è stretta ($f \text{ undefined} = \text{undefined}$)
2. $fa = b$
3. $f(gxy) = hx(fy)$

Allora vale l'identità:

$$f \circ \text{foldr } g \ a = \text{foldr } h \ b$$

Induzione su Liste Parziali

Una lista parziale è una lista che non termina con il costruttore `[]` ma con una computazione divergente (`undefined`). Il principio di induzione su liste parziali richiede di dimostrare:

- **Caso base:** $P(\text{undefined})$ è vera.
- **Passo induttivo:** Se $P(xs)$ è vera (con xs parziale), allora $P(x : xs)$ è vera.

Valori e Forme Normali

Per capire a fondo la *laziness*, dobbiamo tornare al λ -calcolo e definire cosa sia un "valore", ovvero un termine che non necessita di ulteriori riduzioni. Esistono diversi livelli di valutazione:

- **Normal Form (nf):** Un termine senza alcun β -redex (totalmente valutato). Nessuna applicazione $(\lambda x. M)N$ è presente, nemmeno annidata.
 - Es. $I = \lambda x. x$
 - Es. $\lambda x. \lambda y. x$ (Il booleano *True*, o combinatore *K*)
- **Head Normal Form (hnf):** Un termine nella forma $\lambda x_1 x_2 \dots x_n. y \ N_1 N_2 \dots N_m$, dove la "testa" y è una variabile (libera o legata) e non una lambda. I sotto-termini N possono contenere redex.
 - Es. $\lambda x. x \ (\Omega) \rightarrow$ È in *hnf* nonostante contenga il termine divergente Ω , perché la variabile di testa x impedisce l'applicazione di Ω .
- **Weak Head Normal Form (whnf):** Una forma normale "debole" di testa. L'astrazione (o il costruttore) fa da "scudo" e ferma la valutazione: **qualsiasi** termine che inizia con una λ non applicata a nulla è in *whnf*, a prescindere da cosa c'è dentro.
 - Es. $\lambda x. ((\lambda y. y)x) \rightarrow$ È in *whnf* solo perché inizia con un'astrazione. **Attenzione:** questo termine *non* è in *hnf*, perché il suo corpo inizia subito con un redex e non con una variabile.

⚠ Il termine $\lambda y. \Omega$ è in *whnf* (anche se il corpo andrebbe in loop infinito, non lo si esplora).

Essendo basato sulla strategia *call-by-name*, Haskell elegge a "valori" le espressioni in **Weak Head Normal Form (WHNF)**. In pratica:

- Una funzione è un valore. Non si valuta il suo corpo interno finché non le sono stati forniti gli argomenti necessari.

- Un costruttore di dati è un valore in WHNF, indipendentemente dal fatto che i suoi campi siano valutati.

⚡ Gli ingredienti per la laziness sono 3: call-by-name, weak head normal form e sharing

Dove lo **Sharing** specifica che ogni sottoespressione valutata viene memorizzata, in questo modo non viene ricalcolata da capo se ricompare

In questo modo una valutazione lazy non fa mai più riduzioni della valutazione eager corrispondente.

- PRO: infinite valutazioni in meno
- CON: la memoria può riempirsi di tante sottoespressioni non valutate, spossibile space leakage.

Ottimizzazione

📄 Tupling (accoppiamento)

Spesso, le definizioni matematiche ricorsive più naturali portano a ricalcoli inutili o a percorsi multipli e ridondanti sulle strutture dati.

La tecnica del Tupling consiste nel restituire più valori contemporaneamente impacchettandoli in una tupla, permettendo di portare avanti i calcoli in un'unica passata.

La funzione base di fibonacci genera un albero di ricorsione esponenziale (2^n), ricalcolando gli stessi valori innumerevoli volte. Per abbattere la complessità a $O(n)$, usiamo il Tupling per restituire due valori consecutivi della serie:

```
fibT n = fst ( fibPair n )
  where
    fibPair 0 = (0 , 1)
    fibPair n = let (a , b) = fibPair (n -1)
                  in (b , a + b)
```

📄 Parametro Accumulatore

E' una variabile aggiuntiva passata alla funzione ricorsiva che "costruisce" il risultato passo dopo passo.

```
fib n = FibAux n 20 |
  where FibAux n i f fp
        | i == h      = f
        | otherwise = fibAux h (i+1) (f+fp) f
```

L'allocazione della memoria in haskell è implicita per aumentare l'astrazione, oltretutto, i dati sono immutabili.

📄 Il principio chiave

La complessità temporale in un linguaggio lazy non equivale alla valutazione totale dell'input. Il risultato finale può emergere da una valutazione parziale, ignorando grossi blocchi di computazione

che non servono all'output finale.

Esempio differenza calcolo computazionale eager vs lazy

```
-- Inserisce un elemento y nell'ordine corretto in una lista già ordinata
ins y (x:xs)
  | y <= x  = y : x : xs
  | otherwise = x : ins y xs

-- Applica l'inserimento ricorsivamente per ordinare l'intera lista
insSrt (x:xs) = ins x (insSrt xs)
insSrt []     = []

-- Trova il minimo prendendo la testa (head) della lista ordinata
min = head . insSrt
```

Se scrivessimo `min = head . insSrt` in C o in Java, sarebbe considerato un errore di design imperdonabile. Ordinare l'intera lista con l'Insertion Sort costa (n^2) . Prenderne il primo elemento costa (1) . **Risultato:** abbiamo trovato il minimo in tempo quadratico, quando bastava una semplice scansione lineare da (n) . In un mondo lazy, questo codice è invece perfetto e altamente modulare. Ecco perché la complessità crolla magicamente a (n) :

- La funzione `min` richiede solo `head`, ovvero il primo elemento della lista generata da `insSrt`.
- Il motore di valutazione chiede a `insSrt` di calcolare esclusivamente la prima cella della lista risultante (il primo costruttore :).
- Per fornire quel primo elemento, `insSrt` e `ins` "esplorano" la lista quel tanto che basta per far riaffiorare l'elemento più piccolo in cima.
- Tutto il resto della lista rimane non valutato (sotto forma di `thunk`, una computazione in sospeso). Il lavoro effettivamente svolto dal processore equivale a un'unica passata sull'input.

Il **vantaggio**: Questo paradigma permette di scrivere codice estremamente dichiarativo e modulare. Puoi combinare funzioni generiche (come un sort completo e un estrattore di testa) senza pagare il costo computazionale "ingenuo" della loro composizione, ottenendo la massima astrazione con le prestazioni ottimali.

⚡ Foldl'

```
foldl' f v (x:xs) = y `seq` (foldl' f y xs)
  where y = f v x
```

Differenza tra `foldl` e `foldl'` (Il Caso Patologico)

Normalmente, `foldl'` si usa per evitare problemi di memoria (stack overflow) causati dai thunk giganti di `foldl`. Su liste parziali o infinite si comportano in modo simile (si bloccano o non terminano), ma esiste un caso in cui differiscono nel risultato.

La funzione definita nell'appunto è:

```
f n x = if x == 0 then undefined else 0
```

Il trucco è qui: La funzione accetta un accumulatore `n` e un elemento della lista `x`. Tuttavia, **ignora completamente l'accumulatore** `n` nel suo corpo. Il risultato dipende *esclusivamente* dal valore di `x`.

Svolgimento con `foldl` (Lazy)

```
foldl f 0 [0, 2]
```

Poiché `foldl` è pigra, non calcola i risultati intermedi. Costruisce prima tutta l'espressione:

1. Primo passo: l'accumulatore diventa il thunk `(f 0 0)`
2. Secondo passo: l'espressione finale è `f (f 0 0) 2`

A questo punto, Haskell inizia a valutare la funzione più esterna:

- Chiama `f` passando come `n` l'espressione non valutata `(f 0 0)` e come `x` il valore `2`.
- Sostituisce nel corpo: `if 2 == 0 then undefined else 0`
- Poiché `2 == 0` è Falso, restituisce `0`.

Risultato = 0. Il thunk "velenoso" `(f 0 0)` (che conterrebbe `undefined`) non viene **mai valutato** perché la funzione non ne ha bisogno per decidere il risultato finale

Svolgimento con `foldl'` (Eager)

```
foldl' f 0 [0, 2]
```

Poiché `foldl'` è rigorosa, **esige** che l'accumulatore sia valutato a ogni singolo passaggio.

1. **Primo passo (x = 0):** `foldl'` calcola il nuovo accumulatore forzando la valutazione di `f 0 0`.
2. Valuta: `if 0 == 0 then undefined else 0`.
3. Poiché `0 == 0` è Vero, il risultato è `undefined` (che in Haskell rappresenta un errore, un crash o la non-terminazione, indicato formalmente con `⊥`).

Appena il programma tenta di valutare `undefined`, l'esecuzione si interrompe istantaneamente. Il secondo elemento della lista (`2`) non verrà mai raggiunto.

Risultato = undefined (Eccezione).

Riassunto del Comportamento

- `foldl` sopravvive perché, essendo lazy, "scarta" i calcoli intermedi difettosi se l'ultimo passaggio non ne richiede esplicitamente il valore.
- `foldl'` crasha perché la sua natura strict la obbliga a guardare dentro ogni accumulatore intermedio passo dopo passo, inciampando nell'errore prima di poter arrivare alla fine.

Derivazione Formale di Programmi: Il miracolo di scanl

La funzione `scanl` calcola i risultati parziali di un `foldl` applicato a tutti i prefissi di una lista. La sua specifica formale naïf è:

$$\text{scanl } f \text{ e } xs = \text{map } (\text{foldl } f \text{ e}) (\text{inits } xs)$$

(Esempio: `scanl (+) 0 [1, 2, 3] → [0, 1, 3, 6]`)

Poiché `inits` restituisce n sottoliste e `foldl` le ripercorre tutte, questa specifica costa (n^2) . Possiamo derivare una versione (n) ragionando per induzione sulla lista `xs`:

- Caso base (`[]`)

Partiamo dall'applicazione al caso base:

```
scanl f e []
```

Sostituiamo la definizione iniziale:

```
map (foldl f e) (inits [])
```

I prefissi di una lista vuota sono solo la lista contenente la lista vuota, ovvero `[]`:

```
map (foldl f e) [[]]
```

Applicando la `map`, otteniamo:

```
[foldl f e []]
```

Per definizione di `foldl` sul caso base, questo si riduce a `e`. Il risultato è quindi `[e]`.

↻ Prima equazione derivata

```
scanl f e [] = [e]
```

- Passo induttivo (`x:xs`)

Partiamo dall'applicazione al passo induttivo:

```
scanl f e (x:xs)
```

Sostituiamo la definizione iniziale:

```
map (foldl f e) (inits (x:xs))
```

Sappiamo che i prefissi di `(x:xs)` sono la lista vuota `[]` seguita da tutti i prefissi di `xs` a cui è stata anteposta `x` (cioè `map (x:) (inits xs)`). Quindi l'espressione diventa:

```
map (foldl f e) ([] : map (x:) (inits xs))
```

Applichiamo la `map` esterna, distribuendola sulla testa e sulla coda della lista:

```
(foldl f e []) : map (foldl f e) (map (x:) (inits xs))
```

La testa si riduce banalmente a `e`. Per la coda, sfruttiamo la proprietà di composizione dei funtori (`mp A mp B = mp (A B)`):

```
e : map (foldl f e . (x:)) (inits xs)
```

Il termine composto `(foldl f e . (x:))` applicato a una generica lista `ys` produce `foldl f e (x:ys)`.

Ma per la definizione di `foldl`, calcolare un fold partendo dall'accumulatore `e` su `(x:ys)` equivale a ricalcolare il fold sulla coda `ys` usando il **nuovo accumulatore aggiornato** `(f e x)`.

Quindi possiamo riscrivere la composizione come `foldl f (f e x)`. L'espressione diventa:

```
e : map (foldl f (f e x)) (inits xs)
```

Se osserviamo bene, la parte a destra del costruttore `cons` (`:`) corrisponde **esattamente** alla specifica originale di `scanl`, ma con il nuovo accumulatore `(f e x)` e la lista accorciata `xs`.

↻ Seconda equazione derivata

```
scanl f e (x:xs) = e : scanl f (f e x) xs
```

Conclusione

✓ L'algoritmo ottimale

Senza dover inventare nulla a livello imperativo, le rigorose regole della sostituzione matematica ci hanno guidato alla stesura dell'algoritmo.

```
scanl f e [] = [e]
scanl f e (x:xs) = e : scanl f (f e x) xs
```

Strutture dati Infinite

Tipi e oggetti infiniti

Laddove le algebre definiscono il minimo insieme chiuso rispetto ai costruttori (es. i naturali di Peano), un tipo ricorsivo in Haskell è rappresentato da una *co-algebra*, ossia la massima algebra chiusa rispetto ai costruttori (nel caso dei naturali, contenendo $\omega = \text{cc}(\text{cc}(\dots)) = \infty(\dots)$).

Esempi di strutture dati infinite:

- *Naturali*: grazie alla laziness di Haskell, possiamo valutare espressioni come `tre < omega` ottenendo `True` (poiché la valutazione confronta passo passo e si ferma non appena trova la differenza, senza dover calcolare tutto l'infinito).

```
data Nat = Z | S Nat
  deriving (Show, Eq, Ord)
```

- *Solo infinito* ($\omega = \infty$): un tipo senza caso base (manca lo zero). Può rappresentare solo una catena infinita di successori.

```
data NatInf = S NatInf
  deriving (Show, Eq, Ord)
```

- *Liste Classiche*: le classiche liste ricorsive. Avendo il caso base `[]` possono essere finite, ma grazie alla natura co-algebrica di Haskell possono anche essere infinite.

```
data List a = [] | C a (List a)
  deriving (Show, Eq, Ord)
```

- *Liste puramente Infinite* (Stream) Mancando il costruttore per la lista vuota, questo tipo definisce una lista che non può mai terminare (comunemente chiamata Stream).

```
data ListInf a = C a (ListInf a)
  deriving (Show, Eq, Ord)
```

📄 Algebra vs Co-algebra in breve

Algebra: Costruisce verso l'alto a partire da un caso base (minimo punto fisso). Garantisce dati finiti.

Co-algebra: Si "srotola" potenzialmente all'infinito senza bisogno di un caso base (massimo punto fisso). Garantisce la possibilità di dati infiniti.

Produttività

Definizione

Un "oggetto" infinito (come uno stream o una lista lazy) è **produttivo** quando consuma meno di quanto genera. In altre parole, deve sempre essere in grado di produrre il prossimo costruttore in un numero finito di passi, senza bloccarsi.

Esempio di oggetto NON produttivo: `und = (head und) : (tail und)`

Perché non funziona?

Questa espressione entra in un loop infinito (**bottom**) perché cerca di prendere una testa (`head`) da una lista `und` che non ha ancora generato nemmeno il suo primo costruttore (la cella vuota della memoria). Consuma energia per cercare un dato che non ha ancora prodotto.

Hamming Numbers

Il problema classico consiste nel generare tutti i numeri composti esclusivamente dai fattori 2, 3 e 5, disposti in ordine crescente. Formalmente, l'insieme degli **Hamming numbers** si definisce come:

$$= \{2^i 3^j 5^k \mid i, j, k \geq 0\}$$

Si prova a generalizzare la generazione a un intero insieme di generatori. L'equazione degli insiemi diventa:

$$= \{1\} \cup_{g \in G} \{g \cdot n \mid n \in S\}$$

L'unione di molteplici stream può essere espressa elegantemente "foldando" la funzione `union` su una lista di liste. All'interno, mappiamo l'operazione di moltiplicazione (`g`) per costruire gli stream derivati.

Attenzione a foldl

In questi scenari è obbligatorio usare varianti destre come `foldr` o `foldr1`. Applicare un `foldl` su stream di dati infiniti andrà in loop, in quanto non terminerà mai di valutare la condizione! Inoltre `foldr1` ha il vantaggio di non necessitare di un valore di innesco iniziale.

```
union xs@(x:txs) ys@(y:tys)
  | x < y  = x : union txs ys
  | x > y  = y : union xs tys
  | otherwise = x : union txs tys
```

```
hamming gs = hs
  where hs = 1 : foldr union [] (map (\x -> map (x*) hs) gs)
```

L'infinita importanza di essere Lazy

Una lista infinita non è generata istantaneamente, ma può essere vista come il limite delle sue approssimazioni finite.

Per rappresentare una computazione parziale o non definita, introduciamo il simbolo $\perp$ (bottom, letto come `undefined`).

Ad esempio, la lista naturale $[1..]$ è il limite della seguente sequenza di approssimazioni sempre più precise:

$$\perp \rightarrow 1 : \perp \rightarrow 1 : 2 : \perp \rightarrow 1 : 2 : 3 : \perp \dots$$

Una sequenza *converge* a una lista (finita o infinita) se la sua "parte definita" si stabilizza. Al contrario, una sequenza come $\perp, 1 : \perp, 2 : 1 : \perp, 3 : 2 : 1 : \perp \dots$ *non converge* a nessun limite perché offre continuamente informazioni contraddittorie senza mai stabilizzarsi (ossia, nella prima lista il primo elemento è 1, nella seconda è 2 e così via).

L'Ordine Parziale (Flat Domains e Strutture)

Formalizziamo l'idea che "una computazione incrementa l'informazione" definendo un *ordine parziale* $\sqsubseteq$ (si legge "approssima" o "ha meno informazioni di").

- Flat Domain* (Tipi semplici): Per tipi base come `Int` o `Bool`, un valore o non è calcolato ($\perp$) o è calcolato definitivamente.

$$x \sqsubseteq y \iff x = \perp \vee x = y$$

- Prodotti* (Tuple): L'ordine si applica componente per componente.

$$(x, y) \sqsubseteq (x', y') \iff x \sqsubseteq x' \wedge y \sqsubseteq y'$$

e

$$\perp \sqsubseteq (\perp, \perp)$$

- Somme* (Either): Un Left approssima solo un Left, e un Right solo un Right.

$$\text{Left } x \sqsubseteq \text{Left } x' \iff x \sqsubseteq x'$$

- Liste*:

$$[] \sqsubseteq xs \iff xs = []$$

$$(x : xs) \sqsubseteq (y : ys) \iff x \sqsubseteq y \wedge xs \sqsubseteq ys$$

e

$$\perp \sqsubseteq xs$$

- Funzioni*: L'ordine è point-wise (punto per punto).

$$f \sqsubseteq_{A \rightarrow B} g \iff \forall x. f(x) \sqsubseteq_B g(x)$$

Perché si chiama parziale

Si dice "*parziale*" (e non "totale") perché non tutti gli elementi possono essere confrontati tra loro. Per poter dire che $X \sqsubseteq Y$, Y deve contenere tutte le informazioni di X , più eventuali informazioni aggiuntive coerenti.

Se io ho una computazione parziale $A = (1 : \perp)$ e una computazione parziale $B = (2 : \perp)$, nessuna

delle due approssima l'altra. Danno informazioni in conflitto (una dice che la lista inizia per 1, l'altra per 2). Quindi A e B sono *incomparabili*.

L'ordine parziale ci **garantisce** matematicamente che, se una funzione è ben scritta (monotona e continua), man mano che chiediamo più informazioni (salendo nell'ordine $\sqsubseteq$), la funzione restituirà risultati sempre più definiti e coerenti, senza mai contraddirsi, tendendo verso un limite infinito.

Computabilità, CPO e Punti Fissi

Complete Partial Order (CPO)

Un ordine è detto Completo (**CPO**) se ogni catena crescente di approssimazioni $x_1 \sqsubseteq x_2 \sqsubseteq x_3 \dots$ ammette un limite superiore (*sup*) denotato con $\sqcup x$, tale che:

- È un *maggiorante*: $\forall n. x_n \sqsubseteq \sqcup x$
- È il *minimo dei maggioranti*: $\forall n. (x_n \sqsubseteq y) \implies \sqcup x \sqsubseteq y$

Le Proprietà delle Funzioni Computabili

Per poter essere eseguita da un calcolatore, una funzione deve soddisfare due vincoli ferrei:

- **Monotonia**: $x \sqsubseteq y \implies f(x) \sqsubseteq f(y)$. (Più informazione fornisci in input, più informazione definita ottieni in output).
- **Continuità**: $f(\sqcup x) = \sqcup f(x)$. (La funzione analizza una porzione finita di dati alla volta; il risultato sul limite è il limite dei risultati parziali).

⚡ Esempi di funzioni NON computabili

- **Violazione di Monotonia**: $f(\perp) = 0$, e $f(x) = 1$ per $x \neq \perp$. Dato che $\perp \sqsubseteq 1$, dovrebbe essere $f(\perp) \sqsubseteq f(1)$, ma $0 \not\sqsubseteq 1$. Pertanto, nei flat domain, le uniche funzioni non strette ammesse sono le costanti.
- **Violazione di Continuità**: Una funzione $g(xs)$ che restituisce $\perp$ se la lista è finita/parziale, e 1 se la lista è infinita. Non è continua perché applicata alle approssimazioni parziali restituirà sempre $\perp$ (il cui limite è $\perp$), ma applicata al limite restituirebbe 1.

Reticolo Completo ($L, \sqsubseteq$)

Il simbolo $\sqcup$ applicato a questa catena indica il limite, ovvero la "fotografia finale" se potessimo aspettare un tempo infinito. Il supremo $\sqcup$ di quella catena di approssimazioni è la vera lista infinita $[1, 2, 3, \dots]$.

In un **Reticolo Completo**, qualsiasi sottoinsieme di elementi, per quanto caotico o in conflitto, possiede sempre sia un *Supremo* ($\sqcup$, il limite superiore) che un *Infimo* ($\sqcap$, il limite inferiore, ovvero l'informazione massima che hanno in comune).

$\forall A | A \subseteq L, A$ possiede:

- *estremo superiore*: $\sqcup A = \min \{x | \forall a \in A, x \geq a\}$
- *estremo inferiore*: $\sqcap A = \max \{x | \forall a \in A, x \leq a\}$

Teoremi sui Punti Fissi

Sono i teoremi che garantiscono che i nostri programmi ricorsivi abbiano senso matematico:

- **Teorema di Knaster:** In un *reticolo completo*, ogni funzione monotona possiede un minimo e un massimo punto fisso.
- **Teorema di Kleene:** Se f è una funzione *continua* (una funzione che non "salta" alle conclusioni e che preserva i limiti), il *minimo punto fisso* (la soluzione meno vincolata e più generale alla nostra equazione ricorsiva) esiste sempre e si calcola come il limite (supremo) di una sequenza di approssimazioni successive che partono da $\perp$:

$$f^\omega(\perp) = \lim_{n \rightarrow \infty} f^n(\perp)$$

Dove $f^n(\perp)$ significa applicare la funzione f per n volte di fila partendo da $\perp$

Esempio

Prendiamo la definizione della lista infinita di 1:

```
ones = 1 : ones
```

Questa è un'equazione del tipo $x = f(x)$, dove la funzione che "costruisce" la lista è $f(x) = 1 : x$. Vediamo come il **Teorema di Kleene** calcola il punto fisso (cioè la vera lista ones):

- Step 0: $x_0 = \perp$ (Non so cosa sia la lista).
- Step 1: $x_1 = f(\perp) = 1 : \perp$ (So che inizia per 1, ma poi non so).
- Step 2: $x_2 = f(f(\perp)) = 1 : 1 : \perp$ (So che inizia con due 1).
- Step 3: $x_3 = f(f(f(\perp))) = 1 : 1 : 1 : \perp$

La catena di approssimazioni è:

$$\perp \subseteq (1 : \perp) \subseteq (1 : 1 : \perp) \subseteq (1 : 1 : 1 : \perp) \dots$$

Il limite supremo ($\sqcup$) di questa catena infinita è la lista infinita $[1, 1, 1, 1, \dots]$. Haskell esegue esattamente questi passi di approssimazione: grazie alla **lazy evaluation**, srotola la catena di Kleene solo quando gli chiedi esplicitamente un nuovo elemento (ad esempio quando usi `take 3 ones`, il motore si ferma allo Step 3).

Insiemi Induttivi: Le Algebre

Gli insiemi induttivi definiscono le classiche strutture dati finite, come i numeri naturali standard, gli alberi o le liste tradizionali.

- **Definizione "dal basso":** Un insieme induttivo viene costruito partendo dal nulla (solitamente l'insieme vuoto o i casi base).
- **Costruttori:** L'insieme è realizzato definendo i costruttori, che sono le "fabbriche" che creano oggetti di tipo A . In Haskell, questi corrispondono ai *Data Constructor* (es. `[]` e `:` per le liste, o `Z` e `S` per i naturali).

- **Minimo Punto Fisso ()**: L'insieme A è definito matematicamente come il **minimo insieme chiuso** rispetto all'applicazione dei suoi costruttori. Essendo il minimo punto fisso di un'operazione di costruzione, non contiene "elementi extra" (come gli infiniti): ogni elemento $a \in A$ è necessariamente il risultato di un'applicazione **finita** di costruttori a partire dalle costanti di base.

I Principi Associati:

- **Logica (Induzione)**: Per ogni insieme induttivo A , esiste un principio di induzione che permette di dimostrare in modo formale delle proposizioni su A .
- **Informatica (Iterazione/Ricorsione)**: Computazionalmente, a questo insieme A è associato un principio di ricorsione o iterazione, che serve per definire funzioni che "consumano" l'insieme (nella forma $f : A \rightarrow B$). Questo perché un dato induttivo è fatto per essere smontato e ridotto a un risultato.

Insiemi Co-induttivi: Le Co-Algebre (L'Infinito)

Gli insiemi *co-induttivi* rovesciano completamente la prospettiva e permettono di trattare strutture potenzialmente infinite (come gli Stream in Haskell).

- **Definizione "dall'alto"**: Non si parte dal vuoto per costruire, ma si parte da un "universo" di tutte le possibilità (che include alberi infiniti, liste senza fine, ecc.).
- **Distruttori (o Eliminatori)**: L'insieme è realizzato definendo le regole che permettono di riconoscere i valori che **non** vi appartengono. Invece di costruttori, usiamo "osservatori" (es. `head` e `tail`).
- **Massimo Punto Fisso ()**: L'insieme A è il **massimo insieme chiuso** rispetto alle operazioni di "eliminazione". Computazionalmente, se un elemento è scorretto (cioè appartiene a A), ce ne accorgeremo in un numero **finito** di applicazioni dei distruttori. Se un elemento resiste all'infinito alle nostre osservazioni, allora fa parte dell'insieme valido.

I Principi Associati:

- **Logica (Co-induzione)**: Per dimostrare proprietà (come l'equivalenza tra due stream infiniti), si usa il principio di co-induzione (o bisimulazione). Due oggetti sono equivalenti se producono sempre le stesse risposte per ogni possibile osservazione finita.
- **Informatica (Co-iterazione/Co-ricorsione)**: Al posto di consumare dati, qui definiamo funzioni "produttrici" nella forma $f : B \rightarrow A$. Un esempio è la funzione `unfold` o `iterate`, che partendo da un *seme* di tipo B , costruisce (o "srotola") un oggetto di tipo A (la lista infinita).

Esempi di Principi di Induzione Computazionale

Nella pratica funzionale, l'induzione e la ricorsione si manifestano attraverso l'uso del pattern matching e delle funzioni di ordine superiore (come le *fold*).

Tipi induttivi SENZA costruttori ricorsivi

In questi tipi la computazione finisce subito, non c'è ricorsione, ma il principio si applica per distinguere i casi.

1. **Either:** Una funzione $f : \text{either } A \ B \rightarrow C$ è definibile a partire da due funzioni $g : A \rightarrow C$ e $h : B \rightarrow C$. Considerando $x \in \text{either } A \ B$, avremo che f esegue $g(x)$ se x è `Right(a)`, oppure esegue $h(x)$ se x è `Left(b)`.
2. **Tuple:** Per il tipo prodotto (A, B) , una funzione verso C si realizza semplicemente partendo da una funzione che accede ai singoli elementi.
3. **Booleani:** Simile a Either. Per calcolare da un booleano a un tipo C , bastano due valori costanti (o funzioni senza parametri) in C (chiamiamole g e h). Se x è `True` restituiamo g , se `False` restituiamo h . (Questo è l'operatore `if-then-else`).

Tipi induttivi CON costruttori ricorsivi

Qui l'induzione matematica tradizionale prende vita sotto forma di funzioni che si richiamano iterativamente.

1. Numeri Naturali:

- **L'iterazione di base (`iter`):** Per consumare un numero naturale, definiamo un caso base e un passo.

$$\text{iter } f \ b \ 0 = b$$

$$\text{iter } f \ b \ n = f (\text{iter } f \ b \ (n - 1))$$
Spiegazione Integrata: Questo comportamento è strutturalmente analogo ai **Numerali di Church** nel lambda calcolo. In Church, un numero n non è un dato, ma un iteratore puro: $n \ f \ b$ significa applicare la funzione f per n volte alla base b .
- **La ricorsione primitiva (`rec`):** Solitamente, il principio di ricorsione è più potente e generico dell'iterazione pura, perché la funzione di step f riceve in input anche il numero n attuale, non solo il risultato dell'accumulatore:

$$\text{rec } f \ b \ 0 = b$$

$$\text{rec } f \ b \ n = f \ n (\text{rec } f \ b \ (n - 1))$$

2. Liste:

Il principio di iterazione sulle liste è esattamente la funzione `foldr` (fold-right) di Haskell.

$$\text{iter } f \ b \ [] = b$$

$$\text{iter } f \ b \ (x : xs) = f \ x (\text{iter } f \ b \ xs)$$

Spiegazione Integrata: La `foldr` smonta la lista (il caso base vuoto diventa `b`, e l'operatore `:` viene sostituito dall'operatore `f`). È la traduzione computazionale esatta dell'induzione sulle liste.

L'Isomorfismo di Curry-Howard (Programmi = Dimostrazioni)

Una dimostrazione per induzione è la controparte logica dell'iterazione e della ricorsione. Ma in che senso?

In logica costruttiva, una dimostrazione è letteralmente un programma:

- Un teorema per induzione richiede un caso base (prova $P(0)$) e un passo induttivo (se ho una prova di $P(n)$, so costruire una prova di $P(n + 1)$).
- Informatica e logica coincidono: una dimostrazione per induzione non è altro che una **funzione a valori logici**. Questa funzione riceve in input la prova $P(0)$ e la funzione di trasformazione $P(n) \implies P(n + 1)$, e restituisce come output un oggetto logico universale, ovvero una prova per ogni n ($\forall n. P(n)$).

Proprio come `foldr` prende un caso base `b` e una funzione di step `f` per macinare un'intera lista, il sistema di tipi di linguaggi funzionali avanzati processa una prova matematica macinando il passo base e

il passo induttivo. Dimostrare un teorema significa, fisicamente, compilare con successo una funzione ricorsiva il cui tipo di ritorno è il teorema stesso.

Coinduzione e Bisimulazione

Perché ci serve la Coinduzione?

Nella programmazione classica, per dimostrare che due funzioni sulle liste fanno la stessa cosa, si usa il

Principio di Induzione:

1. Dimostri che le funzioni si comportano allo stesso modo sulla lista vuota `[]` (Caso Base).
2. Assumi che funzionino su una coda `xs` e dimostri che funzionano su `x:xs` (Passo Induttivo).

⚡ Il problema dell'Infinito

Sulle liste infinite (Stream) come `ones = 1 : ones`, l'induzione standard **fallisce miseramente**. Non esiste la lista vuota `[]`, quindi manca il caso base! Non potendo mai "smontare" la lista fino in fondo, l'induzione non può partire.

Per dimostrare l'uguaglianza tra oggetti infiniti, dobbiamo usare il concetto duale: la **Coinduzione**. Invece di costruire dal basso, la coinduzione osserva "dall'alto", un pezzo alla volta.

Il Concetto di Bisimulazione

La coinduzione si basa su una tecnica chiamata **Bisimulazione**.

Pensa alla bisimulazione come a un "gioco di osservazione" tra due scatole nere che producono valori all'infinito. Se, ad ogni singolo passo, entrambe le scatole sputano fuori lo stesso identico valore e si preparano per il passo successivo nello stesso identico modo, per noi quelle due scatole sono equivalenti.

📄 Definizione Formale di Bisimulazione

Una relazione R tra due liste xs e ys (scritta $xs R ys$) è una **Bisimulazione** se, ogni volta che osserviamo un passo di computazione, vale una delle seguenti condizioni:

1. Entrambe le liste sono indefinitamente bloccate ($xs = \perp$ e $ys = \perp$), oppure sono entrambe vuote ($xs = []$ e $ys = []$).
2. Se si valuta un elemento, entrambe producono una testa e una coda: $xs = x : xs'$ e $ys = y : ys'$. Affinché la bisimulazione regga, deve accadere che:
 - Le teste sono identiche: $x = y$
 - Le code continuano a rispettare la relazione: $xs' R ys'$

Il Principio di Coinduzione

Il Principio di Coinduzione è il teorema che ci autorizza a usare la bisimulazione come strumento di dimostrazione.

✓ Teorema

Due liste infinite xs e ys sono rigorosamente e matematicamente **uguali** ($xs = ys$) se e solo se esiste una bisimulazione R tale per cui la coppia (xs, ys) appartiene a R .

In parole povere: Se riesci a trovare anche solo una singola relazione R (che inventi tu) che dimostra che le due liste avranno sempre la stessa testa e che le loro code si comporteranno sempre secondo le regole di R , allora hai dimostrato che le due liste sono identiche fino all'infinito.

Esempio Pratico: Dimostrazione per Coinduzione

Vogliamo dimostrare formalmente questo teorema sulle liste infinite in Haskell:

$$\text{map } f (\text{iterate } f x) = \text{iterate } f (f x)$$

(Nota intuitiva: Entrambe le funzioni generano la sequenza $[f(x), f(f(x)), f(f(f(x))), \dots]$. Vogliamo provarlo matematicamente).

Passo 1: Costruire la Relazione R

Inventiamo una relazione R che contenga la struttura generale dell'uguaglianza che vogliamo dimostrare.

Definiamo che due liste sono in relazione R se hanno la forma:

$$[\text{map } f (\text{iterate } f k)] \quad R \quad [\text{iterate } f (f k)]$$

_(Nota che ho sostituito x con k per generalizzare: la relazione deve valere per qualsiasi elemento generico).

Passo 2: Analizzare la parte Sinistra (LHS)

Prendiamo il membro sinistro e "srotoliamo" le definizioni delle funzioni di un singolo passo:

```
map f (iterate f k)
-- Per definizione di 'iterate', (iterate f k) equivale a (k : iterate f (f k))
= map f (k : iterate f (f k))
-- Per definizione di 'map', applico la f alla testa e alla coda
= f k : map f (iterate f (f k))
```

Passo 3: Analizzare la parte Destra (RHS)

Ora facciamo lo stesso con il membro destro:

```
iterate f (f k)
-- Per definizione di 'iterate', metto l'argomento in testa e riapplico iterativamente f
= f k : iterate f (f (f k))
```

Passo 4: Verifica della Bisimulazione

Mettiamo a confronto i risultati di LHS e RHS che abbiamo appena calcolato:

- **LHS:** `f k : map f (iterate f (f k))`

- **RHS:** `f k : iterate f (f (f k))`

Verifichiamo le due regole della bisimulazione:

1. **Le teste sono uguali?** Sì, entrambe le liste hanno `f k` come testa.

2. **Le code appartengono alla relazione R ?** Le due code sono:

- Coda LHS: `map f (iterate f (f k))`

- Coda RHS: `iterate f (f (f k))`

Se guardi la struttura della nostra relazione iniziale R , sostituendo il vecchio k con un nuovo elemento z (dove $z = f k$), le due code assumono esattamente la forma: `map f (iterate f z)` e `iterate f (f z)`.

✓ Conclusione della Dimostrazione

Poiché le teste sono identiche e le code ricadono esattamente nella stessa relazione R di partenza, R è confermata essere una bisimulazione valida. Pertanto, per il Principio di Coinduzione, l'equazione originale è vera.

Definizione dei Numeri di Ulam

I numeri di Ulam, indicati con la successione $\{u_n\}_{n \in \mathbb{N}}$, sono definiti matematicamente in questo modo:

- $u_1 = 1$
- $u_2 = 2$
- u_{n+1} è il minimo numero che può essere scritto in modo unico come somma di due elementi distinti appartenenti all'insieme dei numeri di Ulam precedenti $\{u_1, \dots, u_n\}$.

La sequenza generata è la seguente: 1, 2, 3, 4, 6, 8, 11, 13, 16, 18, 26, 28, 36..

Per capire la logica dell'unicità, prendiamo come esempio il numero 5. Esso non è un numero di Ulam perché può essere ottenuto in due modi diversi usando i numeri di Ulam precedenti (1, 2, 3, 4):

$5 = 1 + 4 = 2 + 3$. Essendo la somma non unica, il 5 viene scartato.

Funtori, applicazioni e monadi

Classe Functor

Concetto Intuitivo: Modificare il contenuto di una scatola

⚡ Un Functore ti permette di prendere una normale funzione pura $f : A \rightarrow B$, come `+1` (che lavora su valori semplici, fuori dal contesto, e per contesto intendiamo una struttura dati come le liste) e applicarla al valore che si trova *dentro* la scatola, **senza rompere o alterare la forma della scatola stessa**.

Definizione Formale

La classe `Functor` richiede l'implementazione della funzione `fmap`. Il suo scopo è sollevare (*lift*) una funzione normale portandola al livello del contesto protetto.

```
class Functor t where
  fmap :: (a → b) → t a → t b
```

Il diagramma di commutazione categoriale garantisce che l'applicazione della funzione mantenga intatta l'architettura della struttura, modificando esclusivamente i valori interni:



Istanze ed Esempi Facili

1. **Maybe (Computazioni che possono fallire):** Se la scatola contiene un valore (`Just x`), `fmap` applica la funzione a `x`. Se la scatola è vuota (`Nothing`), non c'è nulla da modificare, quindi restituisce direttamente `Nothing`.

```
instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just x) = Just (f x)
```

Esempio: `fmap (+1) (Just 5)` produce `Just 6`. Se invece facciamo `fmap (+1) Nothing`, otteniamo `Nothing`.

2. **Liste []:** Mappare una funzione su una lista significa applicarla a ogni singolo elemento, mantenendo la lista della stessa identica lunghezza. Coincide con la funzione `map`.

```
instance Functor [] where
  fmap = map
```

Esempio: `fmap (*2) [1, 2, 3]` produce `[2, 4, 6]`.

3. **Alberi Binari (Tree):** Mostra come `fmap` operi ricorsivamente preservando la forma esatta del grafo:

```
data Tree a = Leaf a | Node a (Tree a) (Tree a)

instance Functor Tree where
  fmap f (Leaf x) = Leaf (f x)
  fmap f (Node x l r) = Node (f x) (fmap f l) (fmap f r)
```

Le Leggi dei Funtori

Qualsiasi istanza valida della classe `Functor` deve soddisfare rigorosamente due leggi algebriche. Se queste leggi vengono violate, il comportamento del codice diventa imprevedibile all'interno delle ottimizzazioni del compilatore.

1. **Identità:** Mappare la funzione identità `id` su un funtore non deve alterare in alcun modo il funtore.

```
fmap id == id
```

2. **Composizione:** Mappare la composizione di due funzioni deve equivalere a comporre le due mappature separate.

```
fmap (f . g) == fmap f . fmap g
```

Classe Applicative

Motivazione: Il Limite dei Funtori

I funtori classici mostrano un limite strutturale insuperabile quando si tenta di applicare funzioni multi-argomento (funzioni parzialmente applicate) a valori inseriti in contesti distinti.

Se consideriamo ad esempio la funzione di addizione `(+) :: Int → Int → Int` e proviamo a mapparla su un contesto `Maybe Int`:

```
fmap (+) (Just 5) -- Risultato: Just (+5) :: Maybe (Int → Int)
```

Otteniamo un risultato strano: `Just (+5)`. Abbiamo ottenuto una funzione bloccata dentro una scatola nel contesto `Maybe`. Se ora volessimo applicare questo `Just (+5)` a un secondo valore `Just 3`, la funzione `fmap` standard fallirebbe perché la sua firma richiede una funzione pura come primo argomento, non una funzione avvolta in un contesto.

Per colmare questo divario, si introduce la classe `Applicative`, che modella l'applicazione contestuale.

Definizione della Classe

```
class Functor t => Applicative t where
  pure  :: a -> t a
  (<*>) :: t (a -> b) -> t a -> t b
```

- `pure`: `a` nel più piccolo contesto `t`, `t` lo si può sostituire con il contesto in cui si trova ad esempio `[a]` $\Rightarrow$ inserimento del contesto nelle liste
- `<*>` (si legge *apply*): Prende una funzione all'interno di un contesto e un valore all'interno di un contesto, producendo il risultato finale all'interno del medesimo contesto.

Istanze ed Effetti Computazionali

1. **Maybe (Propagazione dei Fallimenti):** Se la funzione o l'argomento sono `Nothing`, l'intera computazione fallisce immediatamente.

```
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  (Just f) <*> mx = fmap g mx -- mx è un oggetto maybe
```

Esempio: `pure 5 = Just(5)`

Esempio: `fmap (+) [1,2,3]` mi restituisce una lista di funzioni che aspettano un parametro, ossia `[(+1), (+2), (+3)]` e rimane nel contesto delle liste. Quindi la `fmap` non è più utilizzabile perché richiede che le funzioni siano fuori contesto. Mentre, `fmap (+2) [1,2,3]` restituisce `[3, 4, 5]` quindi non serve eseguire altro.

Esempio:

```
pure (+) <*> (Just 2) <*> (Just 3)
(Just (+)) <*> (Just 2) <*> (Just 3)
(fmap (+) (Just 2)) <*> (Just 3)
(Just (+2)) <*> (Just 3)
fmap (+2) (Just 3)
Just (2 + 3)
```

Esempio:

2. **Liste [] (Calcolo Non-Deterministico):** L'applicazione applicativa sulle liste standard genera il prodotto cartesiano completo di tutte le possibili applicazioni delle funzioni della prima lista agli elementi della seconda.

```
instance Applicative [] where
  pure x = [x]
  fs <*> xs = [f x | f <- fs, x <- xs]
```

Esempio: `[(+1), (*2)] <*> [1, 2]` produce `[1+1, 2+1, 1*2, 2*2]`, ovvero `[2, 3, 2, 4]`.

Esempio:

```
pure (*) <*> [1, 2] <*> [3, 4]
[(*)] <*> [1, 2] <*> [3, 4]
[(+1), (*2)] <*> [3, 4]
[(3*1), (4*1), (3*2), (4*2)] -- tutti i risultati
-- "non deterministicamente"
-- Non è zipWith!
```

3. **ZipList (Combinazione Parallela):** Se si desidera un comportamento di accoppiamento posizionale (stile `zipWith`), si utilizza il tipo wrapper `ZipList` tramite la dichiarazione `newtype`.

```
newtype ZipList a = Z [a]

instance Applicative ZipList where
  pure x = Z (repeat x)
  Z fs <*> Z xs = Z (zipWith (\f x -> f x) fs xs)
```

Nota: `pure` per `ZipList` deve generare uno stream infinito dello stesso elemento (`repeat x`) per fare in modo che l'operazione di `zipWith` non si interrompa prematuramente quando combinata con liste di

lunghezza arbitraria.

Le Leggi degli Applicativi

Gli applicativi devono rispettare quattro leggi fondamentali per preservare la coerenza algebrica del calcolo dei programmi:

1. Identità:

```
pure id <*> x == x
```

2. **Omomorfismo:** Applicare una funzione pura a un valore puro all'interno del contesto tramite `pure` deve equivalere a calcolare l'applicazione fuori dal contesto e poi applicare `pure` al risultato.

```
pure f <*> pure x == pure (f x)
```

Esempio:

```
pure((+2) 1) = pure(+2) <*> pure (1)
pure(3) = Just(+2) <*> Just(1)
pure(3) = fmap (+2) Just (1)
pure(3) = pure(3)
```

3. **Interscambio:** Quando si applica una funzione contestuale a un valore puro, l'ordine con cui i blocchi vengono inseriti nel contesto non conta.

```
u <*> pure y == pure (\f → f y) <*> u
```

Esempio: definiamo i nostri due elementi

- `u` (la funzione nel contesto): Scegliamo `Just (*2)`
 - `y` (il valore puro): Scegliamo `5`
- La legge ci dice che: `Just (*2) <*> pure 5` **deve essere uguale a** `pure (\f → f 5) <*> Just (*2)`

```
Just (*2) <*> pure 5 = pure (\f → f 5) <*> Just (*2)
Just (*2) <*> Just 5 = Just (\f → f 5) <*> Just (*2)
Just (5 * 2) = fmap (\f → f 5) Just(*2)
Just (5 * 2) = Just ( (\f → f 5) (*2) )
Just (5 * 2) = Just ( (*2) 5 )
Just 10 = Just 10
```

4. **Composizione:** Garantisce che la composizione delle funzioni rimanga associativa anche a livello applicativo.

```
(pure (.) <*> u <*> v) <*> w == u <*> (v <*> w)
```

La Classe di Tipi Monad

📄 La Soluzione Monadica

La monade astrae questi tre pattern disomogenei incapsulando la logica di concatenamento, di propagazione degli errori e di passaggio dello stato all'interno di un'unica infrastruttura astratta. Il programmatore definisce le regole di combinazione una sola volta, mantenendo pulita la struttura del codice principale.

La classe `Monad` eredita l'interfaccia della classe `Applicative`. Essa permette di definire una computazione dotata di effetti collaterali tramite due operatori fondamentali:

```
class Applicative m => Monad m where
  (>=) :: m a -> (a -> m b) -> m b
  (>>) :: m a -> m b -> m b
  return :: a -> m a
  return = pure

-- Operatore di sequenzializzazione accessoria
m >> n = m >= \_ -> n
```

- **return**: Prende un valore puro e lo inserisce nel contesto monadico minimo (coincide nativamente con `pure` di `Applicative`).
- **>= (Bind)**: Rappresenta il nucleo della computazione monadica. Prende un valore inserito in un contesto `m a`, ne estrae il contenuto puro `a` in modo protetto, e lo passa a una funzione `(a -> m b)` che genera il passo computazionale successivo dotato di effetto.

Sfruttando l'astrazione monadica, il valutatore universale di Wadler può essere riscritto in forma definitiva indipendentemente dall'effetto specifico (eccezione, log o stato):

```
evalMonadic :: Monad m => Term -> m Int
evalMonadic (Const a) = return a
evalMonadic (Div t u) = evalMonadic t >= \a ->
  evalMonadic u >= \b ->
  return (a `div` b)
```

La Sintassi Do (Zuccherio Sintattico)

Per evitare l'annidamento visivo delle funzioni lambda collegate tramite l'operatore di bind `>=`, Haskell introduce la notazione `do`. Si tratta di una traslazione puramente sintattica gestita direttamente in fase di compilazione secondo regole di riscrittura formale.

Regole di Traduzione del Blocco Do

1. Un singolo comando isolato all'interno del blocco corrisponde all'azione stessa:

```
do { p } => p
```

2. Due azioni poste in sequenza senza legame di variabile vengono tradotte tramite l'operatore di sequenzializzazione scartante `>>`:

```
do { p; stmts } ⇒ p >> do { stmts }
```

3. L'estrazione di un valore tramite l'operatore `<-` viene convertita esplicitamente in un'applicazione infissa dell'operatore di bind `>>=` combinata con una lambda function:

```
do { x <- p; stmts } ⇒ p >>= \x → do { stmts }
```

Sfruttando la notazione `do`, il valutatore monadico assume una forma che ricalca la programmazione imperativa tradizionale, pur preservando la purezza matematica intrinseca del linguaggio:

```
evalDo :: Monad m ⇒ Term → m Int
evalDo (Const a) = return a
evalDo (Div t u) = do
  a <- evalDo t
  b <- evalDo u
  return (a `div` b)
```

Le Leggi Monadiche

Ogni istanza della classe `Monad` deve soddisfare tre leggi algebriche per garantire la coerenza della composizione e la prevedibilità del refactoring del codice.

1. Identità Sinistra

Inserire un valore puro in un contesto tramite `return` e passarlo a una funzione `f` tramite il bind deve essere identico ad applicare direttamente la funzione `f` al valore puro fuori dal contesto.

```
return x >>= f == f x
```

In notazione Do:

```
do { x' <- return x; f x' } == f x
```

2. Identità Destra

Prendere una computazione monadica `m` e passarla tramite bind a `return` non deve alterare o aggiungere effetti alla computazione originale.

```
m >>= return == m
```

In notazione Do:

```
do { x <- m; return x } == m
```

3. Associatività

Garantisce che il raggruppamento delle operazioni in sequenza non influenzi il risultato finale degli effetti accumulati. Permette di srotolare blocchi di computazione complessi.

```
(m >>= f) >>= g == m >>= (\x → f x >>= g)
```

In notazione Do:

```
do { y ← do { x ← m; f x }; g y } == do { x ← m; y ← f x; g y }
```

Formulazioni Alternative: Kleisli e Join

La semantica delle monadi può essere espressa in modo equivalente utilizzando operatori geometricamente differenti rispetto al bind classico.

La Composizione di Kleisli (>=>)

L'operatore di freccia di Kleisli modella la composizione diretta di funzioni dotate di effetti collaterali (funzioni di tipo $a \rightarrow m\ b$).

```
(>=>) :: Monad m => (a → m b) → (b → m c) → (a → m c)
f >=> g = \x → f x >=> g
```

In matematica (e in Haskell), l'operatore standard di composizione di funzioni è il punto $(.)$. Se hai una funzione $g :: a \rightarrow b$ e una funzione $f :: b \rightarrow c$, puoi comporle in $f . g$ (che ha tipo $a \rightarrow c$). Tuttavia, se le tue funzioni restituiscono un *contesto monadico* (es. $a \rightarrow \text{Maybe } b$ e $b \rightarrow \text{Maybe } c$), il punto standard si rompe: la seconda funzione si aspetta un valore puro b , ma la prima gli sta passando un $\text{Maybe } b$.

L'operatore >=> fa esattamente questo: agisce da "tubo di collegamento" intelligente che estrae il valore dal contesto della prima funzione e lo inietta nella seconda, gestendo automaticamente la logica del contesto (ad esempio, fermandosi se c'è un Nothing).

Se si definisce una monade tramite l'algebra di Kleisli, le tre leggi monadiche si riducono elegantemente agli assiomi di un **Monoide**, in cui l'operatore binario associativo è >=> e l'elemento di identità neutro è la funzione `return`:

```
return >=> f == f      -- Identità Sinistra
f >=> return == f      -- Identità Destra
(f >=> g) >=> h == f >=> (g >=> h) -- Associatività
```

Esempio

Immagina di avere delle funzioni che rappresentano interrogazioni in un database anagrafico. Ogni persona potrebbe non avere un record genitore, quindi usiamo la monade `Maybe`:

```
padre :: Persona → Maybe Persona
madre :: Persona → Maybe Persona
```

Se vogliamo trovare il nonno materno (il padre della madre), in matematica pura vorremmo semplicemente comporre le due funzioni: `padre . madre`.

Tuttavia, il compilatore Haskell si ribellerà. L'operatore punto `(.)` prende l'output esatto della prima funzione e lo incastra nell'input della seconda. Ma la funzione `madre` restituisce un `Maybe Persona` (una "scatola" che può contenere una persona o essere vuota), mentre la funzione `padre` accetta solo una `Persona` "pura". La spina non entra nella presa.

È qui che entra in gioco la **composizione di Kleisli** (`>= >`).

L'operatore `>= >` è un connettore intelligente. Quando scrivi `madre >= > padre`, stai dicendo ad Haskell:

1. Esegui `madre`.
2. Se il risultato è `Nothing` (nessuna madre trovata), ferma tutto e restituisci `Nothing`.
3. Se il risultato è `Just persona`, **estrai** la persona dalla scatola e passala alla funzione `padre`.

Hai appena composto due funzioni con effetti collaterali (il potenziale fallimento) creando una nuova funzione perfettamente valida del tipo `Persona → Maybe Persona`, ignorando del tutto la logica di estrazione dei valori.

La Formulazione tramite `join`

Un terzo approccio (derivato direttamente dalla Teoria delle Categorie) definisce una monade tramite l'accoppiamento di `fmap` e della funzione di abbattimento strutturale `join`.

```
join :: Monad m => m (m a) -> m a
join x = x >>= id
```

La funzione `join` si occupa di "appiattire" due livelli di nesting contestuale concentrico (ad esempio, trasforma un tipo `Maybe (Maybe a)` in un tipo `Maybe a`). Sfruttando questa primitiva, l'operatore di bind classico può essere interdefinito come:

```
m >>= f == join (fmap f m)
```

La Formulazione tramite `join`

Mentre Kleisli ragiona in termini di "tubature" tra funzioni, l'approccio con `join` ragiona in termini di **strutture dati annidate**.

Spesso, lavorando con le monadi, capita di applicare una funzione monadica all'interno di un valore che è già monadico. Il risultato è un contesto dentro un altro contesto (`m (m a)`). L'essenza della monade, da questo punto di vista categoriale, è proprio la sua capacità di "collassare" questi due livelli identici in uno solo, senza perdere informazione semantica.

Esempio Pratico: Liste e Maybe

L'esempio più intuitivo di `join` è sulla monade Lista. Se hai una lista di liste, "appiattirla" significa semplicemente concatenarla. In effetti, per le liste, `join` è esattamente la funzione `concat`.

```
import Control.Monad (join)

-- Esempio con le Liste
```

```
join [[1, 2], [3, 4]]
-- Risultato: [1, 2, 3, 4]
```

Con la monade `Maybe`, il `join` distrugge il livello esterno solo se è ridondante:

```
-- Esempio con Maybe
join (Just (Just 5)) -- Risultato: Just 5
join (Just Nothing)  -- Risultato: Nothing
join Nothing         -- Risultato: Nothing
```

Esempi di Monadi Fondamentali

Per comprendere l'esigenza delle monadi all'interno di un linguaggio funzionale puro come Haskell, si analizza il classico problema della ristrutturazione del codice introdotto da Philip Wadler nel 1996. Si consideri un valutatore di espressioni elementari basato su un tipo di dato algebrico astratto per la divisione intera:

```
data Term = Const Int | Div Term Term
```

Un valutatore puro e basilare si implementa in modo lineare tramite pattern matching:

```
eval :: Term → Int
eval (Const a) = a
eval (Div t u) = eval t `div` eval u
```

Nelle applicazioni reali, questo valutatore minimale deve essere esteso per supportare tre comportamenti tipici del paradigma imperativo: la gestione delle eccezioni, la produzione di un log stringa delle operazioni e il tracciamento dello stato mutabile.

La Monade `Maybe`

Rappresenta la computazione con possibilità di interruzione precoce o fallimento.

```
-- (>=) :: Maybe a → (a → Maybe b) → Maybe b
instance Monad Maybe where
  Nothing >= _ = Nothing
  (Just x) >= f = f x

eval (Const a) = Just a
eval (Div t u) = eval t >= \a →
  eval u >= \b →
  safeDiv a b

safeDiv n m = if m == 0
  then Nothing
  else return (n `div` m)
```

Se un qualsiasi passo intermedio restituisce `Nothing`, il bind interrompe la catena e collapse l'intero blocco in `Nothing` senza valutare le funzioni successive.

Il pattern Log (Monade Writer)

E' di tipo `(s, string)`, non riceve uno stato precedente. Ad ogni iterazione si può modificare la stringa e mostrarla. Ma non puoi modificarla in relazione a ciò che c'era in precedenza

Questo pattern serve per affiancare a una computazione principale un output secondario, tipicamente utilizzato per generare log o accumulare informazioni testuali durante l'esecuzione del programma. Nel tuo codice, hai definito un nuovo tipo `Out a` che racchiude il risultato della computazione `a` e una stringa di log.

Nota: Nel tuo manoscritto c'è un piccolo refuso: hai scritto `instance Monad ST where` prima di definire la monade per `Out`. L'ho corretto qui sotto per mantenere la coerenza del codice.

```
-- Definiamo il tipo che contiene il valore 'a' e la stringa di log
newtype Out a = O (a, String)

-- 1. Istanza Functor: Applica una funzione solo al valore, lasciando il log intatto
instance Functor Out where
  fmap f (O (x, s)) = O (f x, s)

-- 2. Istanza Applicative:
-- 'pure' solleva un valore inserendo una stringa vuota come log iniziale.
-- '<*>' combina i valori applicando la funzione, e concatena le due stringhe di log.
instance Applicative Out where
  pure x = O (x, "")
  (O (f, x)) <*> (O (a, y)) = O (f a, x ++ y)

-- 3. Istanza Monad:
-- Estrae il valore 'a', lo passa alla funzione 'f' per ottenere un nuovo
-- stato 'b', e concatena il log vecchio 'x' con quello nuovo 'y'.
instance Monad Out where
  (O (a, x)) >>= f = let O (b, y) = f a
    in O (b, x ++ y)

-- === Valutatore con Log ===

-- Funzioni di appoggio per la formattazione dei log
line t a = "eval (" ++ show t ++ " " <= " ++ show a ++ ["\n"]

logs (O (v, s)) t = O (v, "eval " ++ show t ++ " " <= " ++ show v ++ ["\n"] ++ s)

-- Valutatore che accumula la traccia dell'esecuzione
evalOut (Const a) = O (a, line (Const a) a)
evalOut (Div t u) = evalOut t >>= \a →
  evalOut u >>= \b →
  logs (pure (a `div` b)) (Div t u)
```

Il pattern State Transformer (Monade State)

E' una memoria simulata, come funzione `s→(a,s)` con:

- `s` che è lo stato precedente
- `(a, s)`: `a` che è il return della funzione nel contesto monadico, `s` che è lo stato nuovo dopo aver applicato nella funzione

Questo pattern serve a passare uno "stato" nascosto attraverso una serie di computazioni. Invece di dover passare esplicitamente una variabile di stato a ogni funzione (cosa molto scomoda in un linguaggio puro come Haskell), la *Monade State* nasconde questo passaggio "dietro le quinte".

Nelle tue note (come illustrato anche dai diagrammi a blocchi), uno *State Transformer* è visto come una scatola che prende in input uno stato `s` e restituisce una tupla con il risultato calcolato `x` e il nuovo stato modificato `s'`.

Basi, Funtore e Applicativo

```
-- Definiamo lo State Transformer come una funzione da uno Stato a (Valore, Nuovo Stato)
newtype ST a = S (State → (a, State))

-- In questo specifico esempio, usiamo un Intero come stato per contare
-- il numero di volte che la funzione è stata chiamata.
type State = Int

-- Funzione di utilità per scartare il costruttore 'S' e applicare la funzione interna
app (S st) x = st x

-- 'tick' genera un nuovo State Transformer che, oltre ad applicare la computazione
-- in input, incrementa di 1 lo stato (side effect) senza alterare il valore 'a'.
tick :: ST Int → ST Int
tick (S st) = S (\s → let (a, s') = st s
                        in (a, s' + 1))

-- 1. Istanza Functor: Modifica il valore 'x' applicando 'f', e propaga lo stato 's'
instance Functor ST where
    fmap f st = S (\s → let (x, s') = app st s
                        in (f x, s'))

-- 2. Istanza Applicative:
-- 'pure' non fa nulla allo stato, lo passa invariato.
-- '<*>' esegue prima 'stf' ottenendo il nuovo stato s', poi esegue 'stx' passando s'.
instance Applicative ST where
    pure x = S (\s → (x, s))

    stf <*> stx = S (\s →
        let (f, s') = app stf s
        let (x, s'') = app stx s'
        in (f x, s''))
```

Definizioni Generiche

Alla fine delle tue note riporti le firme standard per le operazioni della monade *State* generica (dove lo stato può essere di qualsiasi tipo `s`, non solo `Int`).

```
-- Definizione generica dello State Transformer (con stato 's' parametrico)
newtype State s a = ST (s → (a, s))

-- Funzioni principali per manipolare lo stato in Haskell:

get      :: State s s      -- Ispeziona lo stato attuale (lo copia nel valore)
runState :: State s a → s → (a, s) -- Esegue lo State Transformer a partire da uno stato
```

```
put    :: s → State s ()      -- Creazione/Modifica: sovrascrive lo stato
state  :: (s → (a, s)) → State s a -- Crea una computazione State da una funzione pura
evalState :: State s a → s → a  -- Esegue ignorando lo stato finale, rit
```

La Monade State e il Valutatore

Qui entra in gioco l'operatore di bind `>>=`, che concatena le operazioni passando lo stato dalla prima alla seconda in modo invisibile, tipico dei diagrammi in cascata che hai disegnato.

```
-- Istanza Monad:
-- Esegue la prima computazione 'st', prende il risultato 'x' e lo passa a 'f',
-- assicurandosi di passare il nuovo stato aggiornato 's'' alla computazione successiva.
instance Monad ST where
  return = pure
  st >>= f = S (\s → let (x, s') = app st s
                      in app (f x) s')
```

-- === Valutatore con Stato (Contatore) ===

```
-- In questo caso, ogni operazione di divisione scatena un 'tick',
-- incrementando lo stato/contatore di 1 in modo automatico.
eval (Const a) = pure a
eval (Div t u) = eval t >>= \a →
  eval u >>= \b →
  tick (pure (a `div` b))
```

Input / Output (La Monade IO)

L'I/O rappresenta un problema nei linguaggi puramente funzionali perché introduce *side-effect* (il risultato dipende da fattori esterni, non solo dagli argomenti).

Haskell risolve questo problema introducendo un argomento implicito chiamato `World` (lo "stato del mondo"). Il tipo base è:

```
type IO a = World → (a, World)
```

Le espressioni di tipo `IO a` sono chiamate **azioni** (actions).

Azioni Base

- `getChar :: IO Char`: legge un carattere (ritorna un valore).
- `putChar :: Char → IO ()`: scrive un carattere (`()` funge da `void`, significa che c'è solo un side-effect).
- `return :: a → IO a`: fa da ponte tra funzioni pure e azioni impure (ma è a **senso unico**: non c'è redenzione dal mondo impuro a quello puro).

Nota sull'ordine di valutazione: Poiché l'I/O interagisce con il mondo esterno, l'ordine delle operazioni conta. Di conseguenza, la valutazione nella monade IO è **strict** (ad esempio, `do { undefined; return 3 }` valuta a `undefined`).

Costrutto Do

Permette di sequenzializzare le operazioni in modo imperativo. Sotto il cofano, utilizza gli operatori di bind `>>=` e sequenza `>>`.

```
getLine :: IO String
getLine = do
  x <- getChar
  if x == '\n'
  then return []
  else do
    xs <- getLine
    return (x:xs)
```

Programmazione Generica, Monoidi e Foldable

Le classi in Haskell (come Functor e Monad) permettono il **Generic Programming**, ovvero lo sviluppo di funzioni che operano su qualsiasi tipo che soddisfi le proprietà (leggi) di quella classe.

Funzioni Generiche Monadiche

- **mapM**: L'equivalente monadico di **map**. Mappa una funzione che ritorna un'azione monadica su una lista e sequenzializza gli effetti.

```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
```

- **filterM**: L'equivalente monadico di **filter**. Se lo si usa con liste che ritornano scelte non deterministiche (es. `[True, False]`), calcola elegantemente il *powerset* di una lista!

Classe Monoid

Generalizza l'idea di avere un'operazione **associativa** e un'**identità** (es. le liste con `++` e `[]`).

```
class Monoid a where
  mempty :: a
  mappend :: a -> a -> a
  mconcat :: [a] -> a -- default: foldr mappend mempty
```

Esempi di Monoidi

1. **Liste**: `mempty = []`, `mappend = (++)`
2. **Maybe**: Si "distribuisce" mantenendo i valori e scartando i `Nothing`.

```
(Just x) `mappend` (Just y) = Just (x `mappend` y)
```

3. **Interi e Booleani (Wrapper Types)**: Poiché i tipi come `Int` o `Bool` possono essere monoidi in più modi (es. Somma/Prodotto per i numeri, AND/OR per i booleani), Haskell usa tipi wrapper come `All` e `Any` per definirli senza ambiguità.

- `All`: `mempty = All True`, `mappend = (&&)`
- `Any`: `mempty = Any False`, `mappend = (||)`

Classe Foldable

Generalizza l'operazione di collasso (riduzione) di valori in una struttura dati verso un singolo valore, usando le logiche dei Monoidi.

```
class Foldable t where
  fold  :: Monoid a => t a -> a
  foldMap :: Monoid b => (a -> b) -> t a -> b
  foldr :: (a -> b -> b) -> b -> t a -> b
  foldl :: (b -> a -> b) -> b -> t a -> b
```

Design dell'Interfaccia

L'interfaccia di `Foldable` ha moltissime funzioni (es. `length`, `null`, `toList`, `maximum`). Il vantaggio è che **fornisce definizioni di default per quasi tutto**.

Per implementare `Foldable` su una tua struttura dati (come un albero binario), è sufficiente implementare **solo una** tra `foldr` e `foldMap`.

Normalmente si implementa `foldMap`:

```
instance Foldable Tree where
  foldMap f (Leaf x) = f x
  foldMap f (Node l r) = foldMap f l `mappend` foldMap f r
```

Classe Traversable

Generalizza ulteriormente `map`, integrando la gestione di eventuali side-effects o computazioni in contesti applicativi (es. fallimenti con `Maybe`).

```
class (Functor t, Foldable t) => Traversable t where
  traverse :: Applicative f => (a -> f b) -> t a -> f (t b)
```

Esempio: Mappare una funzione che può fallire (`pred` che restituisce `Nothing` se $n \leq 0$) su una lista.

- `traverse pred [1, 2, 3] -> Just [0, 1, 2]`
- `traverse pred [2, 1, 0] -> Nothing` (L'intero attraversamento fallisce se un singolo elemento fallisce).

Linguaggio C

Struttura di un Programma C

Ogni programma inizia sempre la sua computazione dalla funzione `main()`. A differenza di altri linguaggi, in C tutte le funzioni vivono allo stesso livello (non si possono annidare funzioni dentro altre funzioni).

```
#include <stdio.h> // Include le librerie di I/O standard

int x = 0; // Variabile globale

int f() { return x; }
void g(int x) { x++; }

int main(int argc, char* argv[]) {
    // La computazione comincia sempre da qui
    g(x);
    int y = f();
    printf("%d\n", x);
    return 0; // 0 significa "tutto bene"
}
```

Tiny C e Asserzioni Logiche

Per ragionare formalmente sui programmi, si utilizza un sottoinsieme del C ("Tiny C") che ammette solo costrutti basilari (`if`, `while`, sequenze, assegnazioni, variabili intere, l'espressione `e+1` e l'uguaglianza `==`).

La correttezza si basa su tre tipi di asserzioni logiche:

- **REQ (Require / Precondizioni):** Assunzioni sui parametri in ingresso.
- **ENS (Ensure / Postcondizioni):** Condizioni garantite sui risultati.
- **INV (Invarianti):** Condizioni logiche sempre vere prima, durante e dopo l'esecuzione di un ciclo.
- **MOD (Side effects)**

Esempio: La somma

Calcoliamo la somma di due numeri interi positivi usando solo incrementi di 1.

```
int somma(int m, int n) {
    /* REQ: m, n >= 0
     * ENS: ritorna m+n
     */
    int i = 0;
    int s = m;
    while (i != n) {
        /* INV: s = m + i */
        s++;
        i++;
    }
    return s;
}
```

```

    s = s + 1;
    i = i + 1;
}
/* All'uscita dal ciclo vale: INV & (i == n) ⇒ s = m + n */
return s;
}

```

Il "Vero Programmatore C"

In C c'è una totale promiscuità tra comandi (che modificano la memoria) ed espressioni. Tutti i comandi restituiscono un valore. Questo permette di scrivere codice estremamente compatto. La stessa somma vista sopra, scritta da un "Vero Programmatore C" (VPC), diventa:

```

int somma_vpc(int m, int n) {
    int i = 0;
    while (i++ != n) m++; // i++ è post-incremento, viene eseguito dopo la valutazione
    return m;
}

```

Nota di attenzione: questa promiscuità può generare errori subdoli se non si fa attenzione alla differenza tra `=` (assegnazione) e `==` (uguaglianza), ad esempio scrivendo `if (x = 0)`.

Teoremi di Iterazione e Terminazione

Per dimostrare formalmente un programma:

1. **Teorema di iterazione finita:** Se un'invariante φ è vera all'ingresso di un ciclo `while(B)`, e l'esecuzione del corpo del ciclo la conserva, se il ciclo termina, allora all'uscita sarà vera l'asserzione $\varphi \wedge \neg B$.
2. **Terminazione:** Si definisce una funzione di terminazione intera dipendente dalle variabili. Se all'interno del ciclo questa funzione è sempre ≥ 0 e decresce strettamente a ogni iterazione, il ciclo terminerà in un numero finito di passi (poiché non esistono catene infinite decrescenti nei numeri naturali $\mathbb{N}$).

Ricorsione (TinyReC)

Se eliminiamo il `while` e le assegnazioni, otteniamo un frammento in cui si usa solo `if` e funzioni ricorsive (simile al lambda-calcolo).

```

int sommaRecAux(int m, int n, int i) {
    if (i == n) return m;
    return sommaRecAux(m+1, n, i+1);
}

int sommaRec(int m, int n) {
    return sommaRecAux(m, n, 0);
}

```

E la corrispondente versione VPC super-compressa (dove `!n` equivale a `n == 0`):

```
int sommaRecVPC(int m, int n) {
    if (!n) return m;
    return sommaRecVPC(m+1, n-1);
}
```

Esempi Classici

Il Predecessore

Calcolare il predecessore potendo solo sommare 1. Si usano due indici mantenendo l'invariante $i = j + 1$.

```
#include <assert.h>

int pred(int a) {
    /* REQ: a > 0
     * ENS: return a - 1 */
    assert(a > 0); // Valuta la condizione, blocca il programma se fallisce
    int i = 0;
    while (i + 1 != a) i++;
    /* INV: i < a */
    return i;
}
```

Comparazione

Una funzione che restituisce 1 se $m > n$, 0 se $m == n$, e -1 altrimenti. Esistono due versioni: una conta all'indietro (più lenta a causa della funzione predecessore) e una in avanti.

```
// Versione che conta in avanti
int compareTo(int m, int n) {
    /* REQ: m, n >= 0 */
    int i = 0;
    if (m == n) return 0;
    while (42) { // 42 è vero, quindi ciclo infinito
        if (i == m) return -1;
        if (i == n) return 1;
        i++;
    }
}
```

Sintesi di Programmi da Asserzioni Logiche

Si può letteralmente "costruire" il codice partendo dalle specifiche matematiche.

Divisione Intera

Specifiche: trovare q e r tali che $m = qn + r \wedge 0 \leq r < n$.

Invariante naturale: $m = qn + r$. Si inizializza con $q = 0, r = m$.

```
int div(int m, int n) {
    /* REQ: n > 0 */
    int q = 0;
    int r = m;
    while (r >= n) {
        /* INV: m = q*n + r */
        r -= n;
        q++;
    }
    return q; // per avere il resto, basterebbe fare: return r;
}
```

Il Concetto di Puntatore

Una variabile puntatore è una variabile che contiene un indirizzo di memoria, tipicamente l'indirizzo di un'altra variabile. Gli operatori fondamentali sono due:

- **&** (Referenziazione): Data una variabile, ne restituisce l'indirizzo di memoria.
- ***** (Dereferenziazione): Dato un puntatore, permette di accedere (in lettura o scrittura) al valore contenuto in quell'indirizzo.
- **NULL**: È la costante che rappresenta il "puntatore a niente" (nessun indirizzo valido).

Esempio illustrativo:

```
void provaPuntatori() {
    int m = 3;
    int n = 1;
    int p;

    int* x = &p; // x contiene l'indirizzo di p
    int* y = NULL; // y non punta a nulla
    int* z = &m; // z contiene l'indirizzo di m

    y = &m; // ora anche y punta a m
    z = &p; // ora z punta a p

    p = 7;
    (*z)++; // Incrementa il valore puntato da z (cioè p, che diventa 8)
    x++; // Incrementa l'indirizzo contenuto in x (pericoloso!)
}
```

Il tipo Universale: **void ***

***** è un costruttore di tipo: dato un tipo **T**, costruisce **T***. Tra tutti i tipi merita menzione **void***.

- Essendo **void** un tipo "vuoto/inutile", **void*** viene interpretato come il **puntatore a un dato di cui non conosco il tipo**.

- È un puntatore universale compatibile per assegnazione con tutti i tipi puntatore.
- **Restrizione:** Non è possibile applicare l'operatore di dereferenziazione `*x` se `x` è di tipo `void*`, perché il compilatore non sa quanti byte deve leggere né come interpretarli.

I Vettori in Linguaggio C e la Memorizzazione

Tutti i linguaggi imperativi offrono la possibilità di usare vettori (o array), che rappresentano un'astrazione della memoria fisica della macchina. La memoria di un calcolatore può infatti essere vista come un unico enorme array in cui ciascun elemento è accessibile tramite il suo indirizzo (puntatore).

Definizione e Inizializzazione

In C, un vettore statico viene definito specificando il tipo degli elementi e una dimensione costante:

```
T a[K]; // Dove T è il tipo e K è una costante
```

Gli elementi dell'array vanno da `0` a `K-1`. I vettori statici possono essere inizializzati al momento della dichiarazione usando le parentesi graffe `{...}`:

```
char s[4] = {'r', 'o', 'm', 'a'};  
// s[0]='r', s[1]='o', s[2]='m', s[3]='a'  
  
int a[100] = {0};  
// Inizializza tutti e 100 gli elementi a 0
```

Memorizzazione Contigua e "Zona Rossa"

Gli elementi di un array sono memorizzati in **celle contigue di memoria**. Il C non offre alcuna protezione nativa sugli indici: espressioni come `a[-1]` o `a[n]` (dove `n` è la lunghezza) sono sintatticamente lecite e non generano errori di compilazione, ma portano l'esecutore a leggere o scrivere nella **zona rossa proibita**, causando comportamenti imprevedibili, corruzione di variabili adiacenti o l'errore di `Segmentation Fault`.

Lo Strano Caso di Dr. Array & Mr. Pointer

In C, il nome di un vettore è a tutti gli effetti un **puntatore all'indirizzo base** (la cella 0) del vettore stesso. Dopo la dichiarazione `int a[n];`, la variabile `a` è di tipo `int*`. Le parentesi quadre `[]` sono semplicemente un **operatore di de-referenziazione mascherato**. La scrittura:

```
a[i];
```

è del tutto equivalente a:

```
*(a + i);
```

Significa letteralmente: *"salta i posizioni dall'indirizzo base di a e prendi il valore"*.

Passaggio di Parametri Array

Quando si passa un array a una funzione, **non si ricopia l'intero array** (sarebbe enormemente inefficiente), ma si passa il puntatore alla sua base. Di conseguenza, la funzione può modificare direttamente gli elementi dell'array del chiamante (side-effect intenzionale).

Esistono due notazioni equivalenti per i parametri formali:

```
void printv(char a[], int n);  
// oppure  
void printv(char* a, int n);
```

Esempi di Stampa con Aritmetica dei Puntatori

Versione I (Notazione vettore come parametro, aritmetica per lo scorrimento):

```
void printv1(char a[], int n) {  
    for (int i = 0; i < n; i++) {  
        printf("%1c", *(a++)); // Avanza direttamente il puntatore base 'a'  
    }  
    printf("\n");  
}
```

Versione II (Notazione puntatore come parametro, indici per lo scorrimento):

```
void printv2(char* a, int n) {  
    for (int i = 0; i < n; i++) {  
        printf("%1c", a[i]); // Usa l'indicizzazione classica  
    }  
    printf("\n");  
}
```

Flessibilità dell'Approccio C

Poiché un array è un puntatore, possiamo riutilizzare la funzione di stampa per visualizzare solo una porzione del vettore compresa tra due indici `inf` e `sup` semplicemente modificando i parametri di chiamata:

```
printv2(&a[inf], sup - inf);
```

L'Aritmetica dei Puntatori

Il significato dell'operatore di incremento `v++` applicato a un puntatore/array dipende strettamente dal **tipo** di dato puntato e dallo spazio in byte richiesto per memorizzarlo:

- Se `v` è un vettore di `char`, `v++` incrementa l'indirizzo di **1 byte**.

- Se `v` è un vettore di `int`, `v++` incrementa l'indirizzo di **4 byte**.
- Se `v` è un vettore di `long int`, `v++` incrementa l'indirizzo di **8 byte**.

L'espressione `v + i` calcola l'indirizzo saltando esattamente `i * sizeof(Tipo)` byte.

L'Allocazione Dinamica di Memoria (Heap)

Cosa succede se vogliamo che una funzione generi un vettore e lo restituisca al chiamante? Scrivere `int p[n]; return p;` è un gravissimo errore in C: il vettore locale `p` viene distrutto (deallocato) alla fine della funzione, e il chiamante riceverebbe un puntatore a "spazzatura".

Per far sopravvivere i dati, si alloca memoria dinamicamente nell'**Heap** usando la libreria `<stdlib.h>`:

- `void* malloc(int n)`: Alloca `n` byte di memoria.
- `void* calloc(int k, int n)`: Alloca `k * n` byte di memoria e **li azzera**.

Attenzione: Poiché tornano `void*` (puntatore universale), è buona norma usare un cast esplicito, ed è tassativo usare `sizeof` per garantire la portabilità tra macchine diverse.

```
int* v = (int*) malloc(n * sizeof(int));
// oppure
int* v = (int*) calloc(n, sizeof(int));
```

Garbage e Memory Leak: La memoria allocata in questo modo rimane occupata finché non viene esplicitamente liberata dal programmatore chiamando la funzione `free(v)`.

Costruttori di Tipo in C: Struct (Record)

Il C permette di costruire nuovi tipi di dato aggregando informazioni eterogenee tramite le `struct` (record). A differenza dei vettori (sequenze omogenee), le `struct` uniscono tipi diversi, come in un'anagrafica.

Definizione di Tipi con `typedef`

Per evitare di scrivere continuamente `struct Nome`, si usa `typedef` per creare un nuovo nome per il tipo:

```
typedef struct D {
    int anno;
    int mese;
    int giorno;
} data;
```

Accesso ai Campi e Puntatori a Struct

L'accesso ai campi di una `struct` passata per valore avviene tramite la dot notation (`d.giorno`). Tuttavia, per modificare una `struct` all'interno di una funzione, dobbiamo passare il suo puntatore e utilizzare l'operatore freccia `→`, che equivale a deferenziare e accedere al campo `(*d).giorno`.

```

void inserisciData(data *d) {
    do {
        printf("\ninserisci il giorno: ");
        scanf("%d", &(d->g));
        printf("\ninserisci il mese: ");
        scanf("%d", &(d->m));
        printf("\ninserisci l'anno : ");
        scanf("%d", &(d->a));
    } while (!legale(*d));
}

```

Le Liste: Rappresentazione in C

Da un punto di vista algebrico, una lista è definita induttivamente: o è vuota (`[]`) oppure è formata da una testa seguita da una coda (`x:xs`).

In C, non potendo definire una struttura di dimensione infinita, utilizziamo l'escamotage dei puntatori. Una lista è rappresentata da un nodo contenente un valore e un **puntatore al nodo successivo**.

```

typedef struct L {
    int val;
    struct L *next;
} listanode;

typedef listanode* lista;
// Nota: il tipo 'lista' è nativamente un puntatore!

```

Costruttori e Distruttori

Il Costruttore `cons`

La lista vuota è rappresentata dal puntatore `NULL`. Per inserire un elemento in testa, definiamo la funzione `cons`, che **alloca dinamicamente memoria** per un nuovo nodo:

```

lista cons(int x, lista L) {
    lista M = (lista) malloc(sizeof(listanode));
    M->val = x;
    M->next = L;
    return M;
}

```

I Distruttori

Servono per accedere alle componenti della lista. Il "Vero Programmatore C" usa direttamente i puntatori (`L->val`, `L->next`), ma logicamente corrispondono a:

```

int head(lista L) {
    /* REQ: L != NULL */
}

```

```

return L->val;
}

lista tail(lista L) {
    /* REQ: L != NULL */
    return L->next;
}

int isEmpty(lista L) {
    if (L == NULL) return 1;
    return 0;
}

```

Polimorfismo e Strutture Generiche in C

Come possiamo scrivere una coda o un albero che possa contenere interi, stringhe, o record complessi senza dover riscrivere il codice ogni volta? In C non esistono i *Generics* (come in Java o C++), ma si sfrutta il puntatore universale `void *`.

`void *` è compatibile per assegnazione con qualsiasi altro tipo di puntatore. Usandolo come tipo del campo informazione, creiamo una struttura polimorfa.

```

typedef struct BG {
    void *info;      // Può puntare a QUALSIASI dato
    struct BG *left;
    struct BG *right;
} binTreeGNode;

typedef binTreeGNode* binTreeGen;

```

Pro e Contro:

- **Vantaggio:** Estrema flessibilità. Possiamo avere alberi (o pile) eterogenei in cui un nodo contiene un intero e un altro nodo contiene una stringa.
- **Svantaggio:** Perdita totale del controllo dei tipi a tempo di compilazione. Quando si estraggono i dati, il programmatore deve fare un casting esplicito sapendo a priori cosa c'è dentro.

Implementazione di una Pila (Stack) Generica

Le pile (LIFO - Last In First Out) sono perfette per sperimentare il polimorfismo. Vengono implementate come liste singolarmente concatenate, mascherate dietro un "Descrittore di Pila".

```

// Nodo della pila generica
typedef struct S {
    void *val; // Puntatore al dato generico
    struct S *next;
} Snode;

// Descrittore della pila (mantiene il top e la dimensione)
typedef struct {
    Snode* top;
}

```

```

    int numElem;
} stackDescriptor;

typedef stackDescriptor *stack;

// --- Operazioni della Pila ---

stack createEmptyStack() {
    stack S = malloc(sizeof(stackDescriptor));
    S->top = NULL;
    S->numElem = 0;
    return S;
}

int isEmpty(stack S) {
    return S->numElem == 0;
}

void push(stack S, void* el) {
    Snode *tmp = malloc(sizeof(Snode));
    tmp->next = S->top;
    tmp->val = el; // Assegno il dato generico
    S->top = tmp;
    S->numElem++;
}

void pop(stack S) {
    Snode *tmp;
    if (!isEmpty(S)) {
        S->numElem--;
        tmp = S->top; // Salvo per poter fare la free
        S->top = (S->top)->next; // Avanzo il top
        free(tmp); // Evito memory leak
    }
}

void* top(stack S) {
    return (S->top)->val; // Restituisce un void*
}

```